

基于动态视轮廓与连续事件检测的 无人机烟幕干扰协同优化模型

摘要

针对多架无人机协同投放烟幕干扰弹以遮蔽来袭导弹视线的问题，本文建立由三维运动学、动态视轮廓判定、连续事件检测和时空区间覆盖组成的统一建模框架。首先将真目标描述为半径 7m、高度 10m 的完整圆柱体，以导弹为动态观测点构造目标视轮廓；随后计算烟幕球心到导弹—目标有限视线段的最大距离，并以该距离不超过烟幕有效半径作为单云团完全遮蔽判据。在此基础上，利用连续根搜索确定遮蔽区间端点，避免固定时间步长造成的漏检与累计误差。多弹问题采用保守口径：先独立判定每个云团能否完整遮蔽目标，再在时间轴上合并有效区间，不额外计入多个部分遮蔽云团在空间上的联合覆盖。

针对问题一，根据给定策略建立导弹、无人机、烟幕弹及烟幕云团的分段运动模型，得到有效遮蔽区间为 $[8.056445, 9.448088]$ s，有效遮蔽时长为 1.391643s。针对问题二，以航向角、飞行速度、起爆时刻及投放一起爆时间差为决策变量，通过差分进化搜索、临界事件降维和连续精化，得到航向角 5.104610° 、速度 140m/s 的投放方案，有效遮蔽时长为 4.588064s。针对问题三，以三个单弹遮蔽区间并集的测度为目标，构建同航迹多弹时序协同模型，通过临界事件接力使相邻遮蔽区间连续衔接，形成 $[5.194503, 12.812051]$ s 上的连续遮蔽，总时长为 7.617548s。

针对问题四，将三架无人机的决策空间分解为相互独立的单机可达策略集。三组经独立数值优化得到的有效遮蔽区间互不重叠，故联合时长等于三组当前单机最优候选的时长之和，FY1、FY2 和 FY3 的有效遮蔽时长分别为 4.588064s、3.989594s 和 3.163569s，总时长为 11.741227s。针对问题五，建立“单弹候选生成—同机航迹组合—跨导弹任务分配—连续局部精修—连续几何复核”的分层优化模型，允许同一无人机的不同烟幕弹干扰不同导弹。所得高质量可行方案共使用 12 枚烟幕弹，三枚导弹的累计有效遮蔽时长分别为 17.233817s、9.937356s 和 9.105011s，总目标函数值为 36.276185s。

最后，通过轮廓与时间离散精度对比、遮蔽边界连续求根以及完整圆柱高密度离散复核检验模型。60 个关键几何状态均满足完全遮蔽条件，最大视线距离为 9.999637m，表明所得策略与统一遮蔽判据相容，数值结果具有较好的稳定性。

关键词：烟幕干扰；动态视轮廓；连续事件检测；区间覆盖；分层优化

一、问题重述

1.1 问题背景

烟幕干扰通过在来袭武器与被保护目标之间形成具有一定空间尺度和持续时间的遮蔽区域，削弱制导系统对目标的稳定识别。已有研究也从图像特征角度探索烟幕干扰效果评价[8]。无人机具有部署灵活、可在指定航向和速度下快速到达投放区域等特点，但烟幕弹脱离无人机后还要经历抛体运动，起爆后的云团又将持续下沉。因此，投放点、起爆点与有效遮蔽区间之间存在显著的时空耦合。

本题给定三枚导弹、五架无人机及一个圆柱形真目标的初始位置与运动规则，要求在无人机保持定高直线匀速飞行、烟幕弹按规定间隔投放、烟幕云团半径和有效时间受限等条件下，设计从单机单弹到多机多弹、多导弹协同的投放策略。核心任务并非仅使烟幕球覆盖某一条视线，而是保证从动态导弹位置观察时，真目标整体在规定意义下被完全遮蔽。

1.2 问题重述

问题一要求在 FY1 的飞行方向、速度、投放时刻和起爆延迟均已给定时，计算该烟幕弹对导弹 M1 的有效遮蔽时长。该问题属于确定参数下的三维运动学与连续几何判定问题。

问题二要求调整 FY1 的航向、速度以及烟幕弹投放和起爆时序，使单枚烟幕弹对 M1 的有效遮蔽时长尽可能大。该问题在问题一的判定模型上增加连续决策变量，形成带物理可行域约束的非线性优化问题。

问题三允许 FY1 连续投放三枚烟幕弹。三枚烟幕弹共享同一无人机航迹，同时受到相邻投放时刻间隔不小于 1s 的约束，目标是通过合理安排三枚烟幕弹的起爆位置和时序，延长 M1 所受遮蔽区间并集的总长度。

问题四要求 FY1、FY2 和 FY3 各投放一枚烟幕弹干扰 M1。各无人机具有独立的初始位置、航向、速度及投放时序，需要确定三组策略并计算联合有效遮蔽时长。

问题五进一步扩展到五架无人机、三枚来袭导弹和每机最多三枚烟幕弹的协同决策。除连续轨迹参数外，还需确定每枚烟幕弹的干扰对象，因而同时包含连续优化、同机时序约束与跨导弹组合分配。

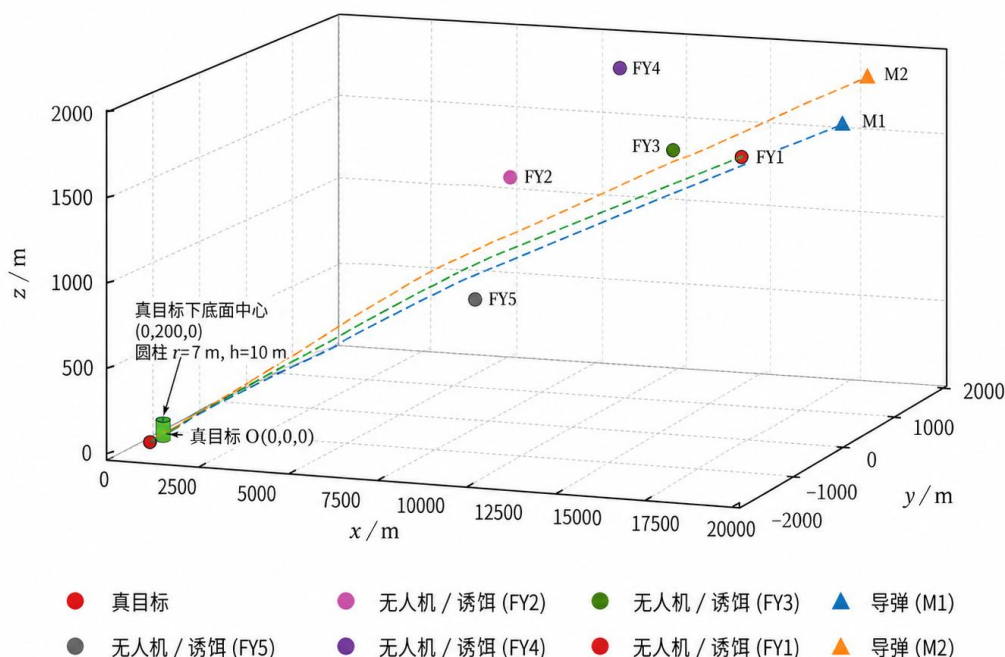


图 1 导弹、无人机及真假目标初始三维空间态势

图 1 给出了三枚来袭导弹、五架无人机以及真假目标的初始三维位置关系。三枚导弹均沿指向假目标原点的直线飞行，该空间态势为后续无人机可达域、投放位置和遮蔽几何分析提供直观基础。

二、问题分析

2.1 统一判定难点

烟幕是否有效首先取决于“完全遮蔽”的数学口径。若仅把目标中心作为判定点，所得条件只能保证目标中心视线被遮挡，不能推出具有有限尺寸的圆柱目标整体不可见；若把导弹—目标连线视为无限直线，又可能把烟幕位于导弹背后或目标之后的情形误判为有效。为此，本文以完整圆柱目标相对于导弹的动态视轮廓为控制边界，并计算烟幕球心到导弹—目标有限线段的距离。该定义同时保留目标尺度、观察方向和空间前后关系。视轮廓随观察点改变而变化的几何性质已有较系统研究[1-2]，其思想适合用于本题动态观察场景。

第二个难点在于遮蔽状态是时间的连续函数，而起效与失效时刻通常不落在预设时间网格上。固定步长扫描虽然计算简便，但会产生与步长同量级的端点误差，甚至漏过持续时间较短的遮蔽事件。连续碰撞检测的基本思想是直接追踪几何接触事件[3]。本文据此构造遮蔽裕度函数，以网格扫描定位符号变化区间，再采用 Brent 方法求解零点[4]，从而得到连续意义下的遮蔽区间。

2.2 各问题的递进关系

问题一用于建立并验证统一运动学—几何判定模型，是后续所有优化问题的计算内核。问题二在该内核外增加四维连续决策变量。由于遮蔽区间长度由若干事件根共同决定，目标函数具有非光滑、分段和多峰特征，传统梯度方法依赖局部光滑性，难以稳定搜索全部可行区域；差分进化对不可导连续优化问题具有较好的适应性[5]，故用于粗搜索，再通过边界结构和一维精化提高端点精度。

问题三的关键不在于三枚烟幕弹单独时长之和，而在于三个遮蔽区间并集的测度。若相邻区间大量重叠，则单弹贡献被重复计算；若区间之间存在较大空隙，则连续防护能力下降。因此，本文先生成单弹候选，并根据数值搜索中出现的“相邻区间首尾接近”结构，将临界接力关系作为三弹时序连续精修的启发式约束。

问题四中三架无人机之间不存在共享航迹约束，可先在各自可达策略集内独立求解单弹优化问题。可达集思想常用于刻画动态系统在约束下能够达到的状态集合[6]。若各单机数值优化候选分别达到其全局最优，且对应有效区间互不重叠，则三机联合时长可达到单机最优时长之和给出的理论上界。本文据此将高维联合问题分解为三个独立低维问题，并在结果部分对“当前单机候选之和”与“严格理论上界”作区分。

问题五同时包含无人机航迹、投放时序和导弹分配。直接对全部连续变量和离散分配变量进行一次性联合优化，维数高且单次精确几何计算代价较大。为兼顾搜索范围与计算精度，本文采用分层策略：先以低成本代理判据生成单弹候选，再在同机航迹约束下组合候选计划，随后求解跨导弹的区间覆盖分配；在确定高价值组合后，对航向、速度和投弹时序进行连续局部精修，最后调用统一连续几何模型复核。该方法得到的是经物理约束与连续几何检验的高质量方案，不据此宣称严格全局最优。

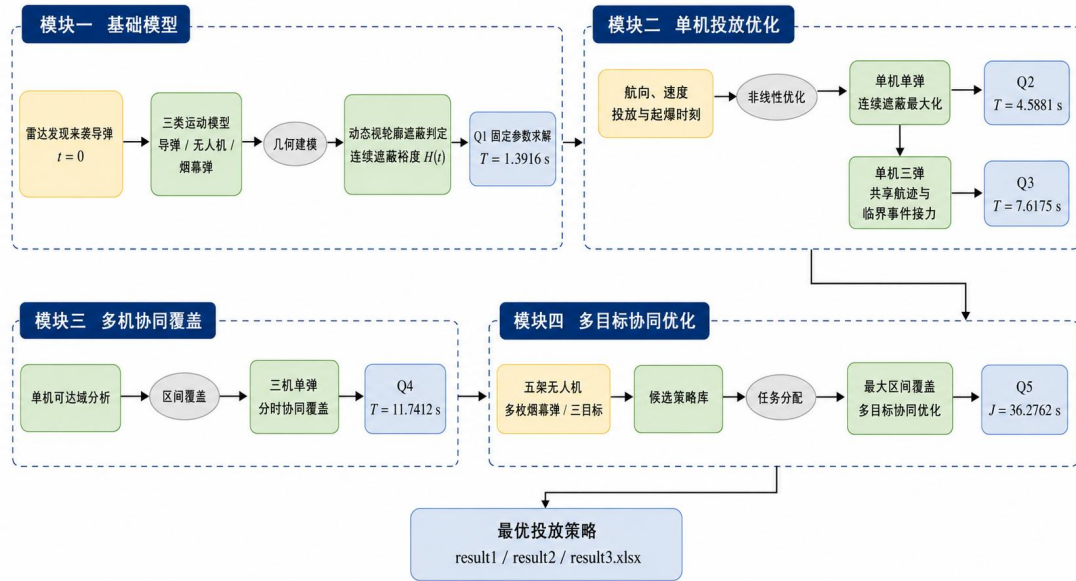


图 2 多无人机烟幕干扰策略的整体建模与求解流程

如图 2 所示，五个子问题共享统一的运动学与遮蔽判定内核，并沿“确定性计算—连续参数优化—时间接力—多机分解—多目标分配”的路径逐步扩展。代理模型只承担候选筛选任务，所有最终结果均返回统一连续模型重新计算。

三、模型假设与符号说明

3.1 模型假设

(1) 导弹自初始时刻起以 300m/s 的恒定速度沿其初始位置指向假目标原点的方向作直线运动。该假设直接对应题目给定的制导运动规则。

(2) 无人机接到任务后瞬时确定航向，并在固定高度以 70~140m/s 的恒定速度作直线运动；不计转弯过渡过程。该假设用于将无人机航迹参数化为航向角和速度。

(3) 烟幕弹脱离无人机后仅受重力作用，忽略空气阻力、风场和自旋；水平方向保持投放瞬间速度，竖直方向作自由落体。计算中取重力加速度 $g=9.8\text{m/s}^2$ 。

(4) 烟幕弹起爆后立即形成半径 10m 的球形有效云团，有效期为 20s，云团只以 3m/s 匀速下沉，半径在有效期内保持不变；不同云团之间不发生合并、扩散或相互作用。

(5) 真目标严格表示为半径 7m、高 10m 的实心圆柱，其整体被烟幕遮蔽才计为有效；只遮蔽目标中心或部分表面不计入有效时长。

(6) 题目未规定最小引信延迟，因此允许投放时刻与起爆时刻相同，即延迟时间可取 0；相邻烟幕弹投放时刻仍满足不小于 1s 的约束。

(7) 同一导弹在不同时间被多枚烟幕独立完整遮蔽时，重叠时间只计一次；问题五以三枚导弹各自有效遮蔽区间并集长度之和作为主目标。

(8) 对于多枚烟幕的空间联合效应，本文采用保守口径：仅当至少一个单独云团能够独立遮蔽完整圆柱目标时，才将该时刻计为有效；不把多个只能遮住目标局部区域的云团进行空间拼接。因而问题三至五的多弹结果在更一般的联合空间覆盖口径下可视为保守可行结果。

(9) 问题五为与结果文件中“干扰的导弹编号”保持一致，每枚实际使用的烟幕弹指定一个主干扰对象，不对其可能同时影响其他导弹的附带收益重复计入。

3.2 主要符号

表 1 统一模型的主要符号及含义

符号	含义	单位
$M_i(t)$	第 i 枚导弹在时刻 t 的位置向量	m
$F_k(t)$	第 k 架无人机在时刻 t 的位置向量	m
$D_{\{kn\}}$	第 k 架无人机第 n 枚烟幕弹的投放点	m
$B_{\{kn\}}$	第 k 架无人机第 n 枚烟幕弹的起爆点	m
$C_{\{kn\}}(t)$	起爆后烟幕云团球心位置	m
θ_k	第 k 架无人机的航向角	rad (结果以 $^\circ$ 报告)
v_k	第 k 架无人机的飞行速度	m/s
$t^d_{\{kn\}}$	烟幕弹投放时刻	s
$t^b_{\{kn\}}$	烟幕弹起爆时刻	s
$\tau_{\{kn\}}$	投放至起爆的时间差	s
$\Gamma_i(t)$	从导弹 M_i 观察真目标得到的动态视轮廓	—
$d_{\{ikn\}}(Q,t)$	第 k 架无人机第 n 枚烟幕云团球心到 M_i — Q 有限视线段的距离	m
$D_{\{ikn\}}(t)$	单枚烟幕对 M_i 动态控制边界的最大有限线段距离	m
$H_{\{ikn\}}(t)$	单枚烟幕对 M_i 的完全遮蔽裕度函数	m
$I_{\{ikn\}}$	第 k 架无人机第 n 枚烟幕弹对 M_i 形成的有效遮蔽区间集合	s
T_i	第 i 枚导弹的累计有效遮蔽时长	s
g	重力加速度，本文取 9.8	m/s ²
T_i^M	第 i 枚导弹到达假目标原点的时刻	s

四、统一运动学与动态视轮廓遮蔽模型

4.1 坐标系与运动学方程

以假目标中心为坐标原点，地面为 xOy 平面，竖直向上为 z 轴正方向。设第 i 枚导弹初始位置为 M_i^0 ，其速度方向由 M_i^0 指向原点，速度大小为 $v_M=300\text{m/s}$ ，则导弹轨迹为

$$\mathbf{v}_i^M = -300 \frac{\mathbf{M}_i^0}{\|\mathbf{M}_i^0\|_2}, \quad \mathbf{M}_i(t) = \mathbf{M}_i^0 + \mathbf{v}_i^M t \quad (4-1)$$

第 i 枚导弹到达假目标原点的时刻为 $T_i^M = \|M_i^0\|/300$ 。后续烟幕有效时间窗均截断在导弹到达原点之前，即只统计 $t \leq T_i^M$ 的遮蔽。

设第 k 架无人机初始位置为 F_k^0 ，航向角 θ_k 从 x 轴正向逆时针计量，则其水平单位方向向量及位置分别为

$$\mathbf{e}(\theta_k) = (\cos\theta_k, \sin\theta_k, 0), \quad \mathbf{F}_k(t) = \mathbf{F}_k^0 + v_k t \mathbf{e}(\theta_k) \quad (4-2)$$

第 n 枚烟幕弹在 $t^d_{\{kn\}}$ 时刻投放，在 $t^b_{\{kn\}}$ 时刻起爆，记 $\tau_{\{kn\}} = t^b_{\{kn\}} - t^d_{\{kn\}} \geq 0$ 。忽略空气阻力后，投放点与起爆点为

$$\mathbf{D}_{kn} = \mathbf{F}_k^0 + v_k t^d_{kn} \mathbf{e}(\theta_k) \quad (4-3)$$

$$\mathbf{B}_{kn} = \mathbf{F}_k^0 + v_k t^b_{kn} \mathbf{e}(\theta_k) - (0, 0, 1/2 g \tau_{kn}^2) \quad (4-4)$$

烟幕起爆后的球心仅沿竖直方向下沉，故在有效期 $t \in [t^b_{\{kn\}}, t^b_{\{kn\}} + 20]$ 内有

$$\mathbf{C}_{kn}(t) = \mathbf{B}_{kn} - (0, 0, 3(t - t^b_{kn})) \quad (4-5)$$

4.2 完整圆柱目标与动态视轮廓

真目标底面中心位于 $(0, 200, 0)$ ，半径为 7m，高度为 10m，因而其空间集合可写为

$$\mathcal{T} = \{(x, y, z) | x^2 + (y - 200)^2 \leq 7^2, 0 \leq z \leq 10\} \quad (4-6)$$

从导弹位置 $M_i(t)$ 观察圆柱目标时，目标外轮廓随观察位置变化。记该轮廓在三维目标表面上的生成点集为 $\Gamma_i(t)$ 。基于圆柱目标的凸性及其投影视轮廓结构，优化内层以上、下圆周作为候选控制边界进行连续极值搜索，以显著降低重复几何计算量；对于侧面母线是否存在更不利点的问题，本文不作解析最优性假设，而在最终方案上对圆柱侧面、顶面和底面进行高密度全边界复核。该处理既避免以单一目标中心替代完整目标，又明确区分了优化近似与最终数值审计。

4.3 有限视线段距离

任取 $Q \in \Gamma_i(t)$ ，导弹至目标点 Q 的视线段参数化为 $L(\lambda) = M_i(t) + \lambda(Q - M_i(t))$ ，其中 $\lambda \in [0, 1]$ 。烟幕球心 $C(t)$ 在该线段上的投影参数为

$$\lambda^* = \text{clip}_{[0,1]} \left(\frac{(\mathbf{C}(t) - \mathbf{M}_i(t)) \cdot (\mathbf{Q} - \mathbf{M}_i(t))}{\|\mathbf{Q} - \mathbf{M}_i(t)\|_2^2} \right) \quad (4-7)$$

其中 clip 算子将投影参数截断到 $[0, 1]$ ，由此排除烟幕位于导弹后方或目标后方却被无限直线误判为有效的情形。球心到有限视线段的距离为

$$d(\mathbf{Q}, t) = \|\mathbf{C}(t) - [\mathbf{M}_i(t) + \lambda^*(\mathbf{Q} - \mathbf{M}_i(t))]\|_2 \quad (4-8)$$

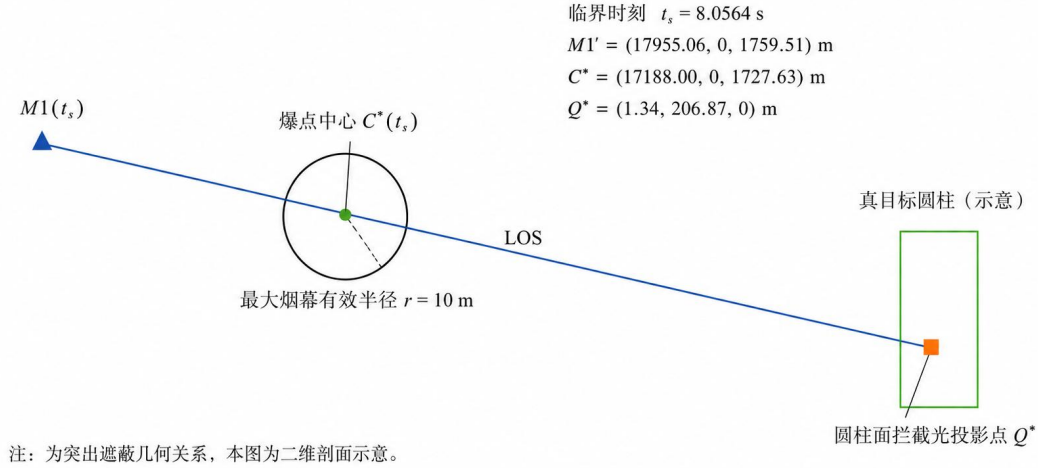


图 3 动态视轮廓条件下烟幕球与导弹视线的临界遮蔽关系

图 3 以问题一的临界状态为例直观展示有限视线段距离判据：当烟幕球心到最不利目标视线段的最短距离恰等于 10m 时，系统处于单云团完全遮蔽的临界状态；距离小于 10m 时则具有正遮蔽裕度。

4.4 完全遮蔽判据

对第 k 架无人机第 n 枚烟幕弹，若要使完整圆柱目标被半径 $R_s=10\text{m}$ 的单个烟幕球独立遮蔽，则所有控制视线段均须与该烟幕球相交。为避免多弹下标歧义，定义该烟幕对导弹 M_i 的动态边界最大距离 $D_{\{ikn\}}(t)$ 及遮蔽裕度 $H_{\{ikn\}}(t)$ 如下。

$$D_{\{ikn\}}(t) = \max_{Q \in \Gamma_i(t)} d_{\{ikn\}}(Q, t), H_{\{ikn\}}(t) = R_s - D_{\{ikn\}}(t) \quad (4-9)$$

在本文动态控制边界模型下， $H_{\{ikn\}}(t) \geq 0$ 表示该单个烟幕满足完整遮蔽条件， $H_{\{ikn\}}(t) = 0$ 对应烟幕开始起效或失效的临界事件。对于多枚烟幕，本文采用保守单云判据：只要存在某一枚烟幕独立满足 $H_{\{ikn\}}(t) \geq 0$ ，即认为该导弹在时刻 t 处于有效遮蔽状态；多个仅部分遮蔽目标的云团不进行空间拼接。若进一步研究多云团空间联合覆盖，应改用 $D_i^{\{\text{multi}\}}(t) = \max_{Q \in \Gamma_i(t)} \min_{k,n} d_{\{ikn\}}(Q, t)$ ，这属于本文模型的可扩展方向。

4.5 连续事件检测与区间合并

对每一枚烟幕弹，在有效时间窗 $[t^b_{\{kn\}}, \min(t^b_{\{kn\}} + 20, T_i^M)]$ 内先以较疏时间网格计算 $H_{\{ikn\}}(t)$ ，定位相邻采样点之间的符号变化；随后对每个异号区间采用 Brent 方法求解 $H_{\{ikn\}}(t) = 0$ 。Brent 方法结合二分、割线和逆二次插值，在保持区间收敛保证的同时具有较高求根效率[4]。将所得根与时间窗端点共同排序，并在相邻根之间取中点判断符号，即可得到该烟幕的精确有效区间集合 $I_{\{ikn\}}$ 。

设导弹 M_i 受到的全部单云有效区间集合为 $\{I_{\{ikn\}}\}$ 。将所有区间在时间轴上取并集，并把相交或相接的区间合并为两两不交的区间集合 U_i ，则累计有效遮蔽时长为

$$T_i = \mu(U_i) = \mu(U_{\{k,n\}} I_{\{ikn\}}) = \sum_{\{a,b\} \in U_i} (b-a) \quad (4-10)$$

其中 μ 表示一维 Lebesgue 测度。该定义确保多个烟幕同时有效时重叠时间只计一次。问题一至四以指定导弹的 T_i 为目标；问题五采用

$$J = \sum_{i=1}^3 T_i \quad (4-11)$$

作为主目标，并同时报告 $\min_i T_i$ 及三枚导弹同时被遮蔽的持续时间，以反映方案的均衡性和协同性。

4.6 求解框架与可行性复核

问题二至五的目标函数均需嵌套调用几何极值与事件求根，计算成本较高。本文将求解分为“搜索—连续精修—正式复核”三层：搜索层使用离散轮廓、较疏时间网格和低成本代理指标快速定位高价值区域；连续精修层围绕候选方案对航向、速度和投弹时序进行局部连续优化；正式复核层使用动态边界连续极值、Brent 求根和区间并集重新计算最终策略。差分进化仅用于连续决策空间的全局区域探索[5]，其输出不能直接视为全局最优证明。数值计算由 Python 环境中的数组运算、优化与求根工具实现。

五、问题一：给定策略下的有效遮蔽计算

5.1 参数代入与烟幕轨迹

FY1 以 120m/s 朝向假目标方向飞行，于 1.5s 投放烟幕弹，3.6s 后起爆，因此 $t_d=1.5s$ 、 $t_b=5.1s$ 。由式（4-3）和式（4-4）得到投放点 $D=(17620,0,1800)m$ ，起爆点 $B=(17188,0,1736.496)m$ 。起爆后云团中心轨迹为 $C(t)=(17188,0,1736.496-3(t-5.1))m$ 。

5.2 连续事件求解

在[5.1,25.1]s 内计算 M1 对应的遮蔽裕度 $H_1(t)$ 。扫描结果显示存在一个连续非负区间；对两端符号变化区间分别求根，得到起效时刻 8.056445490s 和失效时刻 9.448088159s。进一步求解 $\|M_1(t)-C(t)\|=10$ 得到 M1 于 9.389247540s 进入烟幕球、约 9.448088159s 离开烟幕球。因此，在 8.056445490~9.389247540s 内，烟幕位于导弹与目标之间并通过完整视轮廓实现遮挡；此后 M1 短暂进入烟幕球内部，完全遮蔽持续到其离开烟幕球附近。该过程说明有限目标尺度和有限视线段判据对起效时刻具有实质影响。

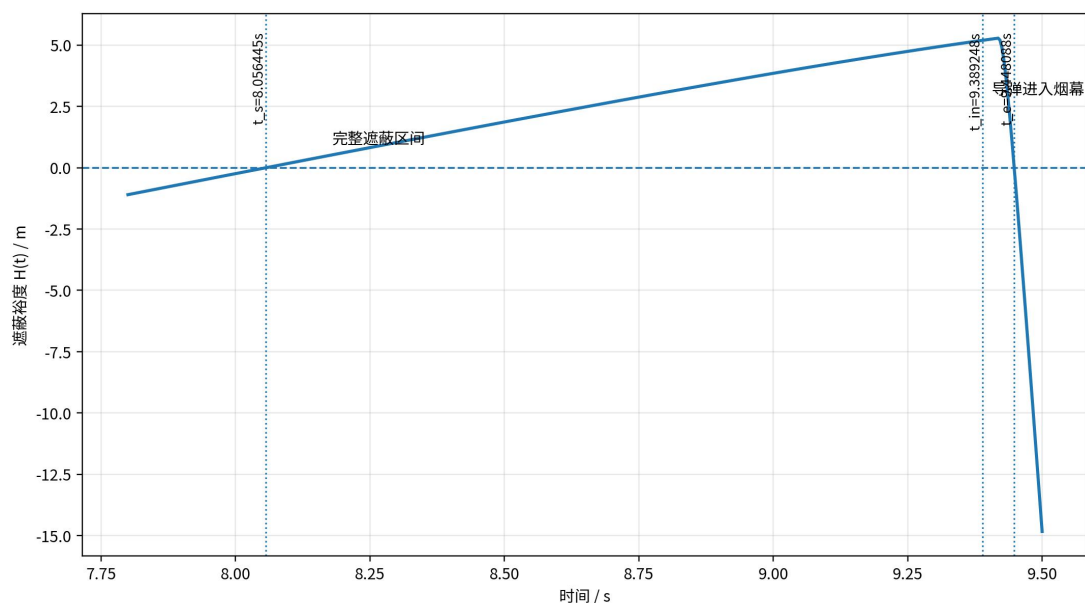


图 4 问题一烟幕完整遮蔽裕度及关键连续事件

图 4 显示遮蔽裕度 $H_1(t)$ 在 8.056445490s 首次越过零轴，并于 9.448088159s 回到零轴，两零点之间即为问题一的连续有效遮蔽区间；其中 9.389247540s 对应 M1 进入烟幕球的关键事件。

表 2 问题一给定策略的运动参数与计算结果

项目	数值
无人机速度	120m/s
投放时刻	1.500000s
起爆时刻	5.100000s
投放点	(17620.000,0.000,1800.000)m
起爆点	(17188.000,0.000,1736.496)m
有效遮蔽区间	[8.056445,9.448088]s
有效遮蔽时长	1.391643s

由表 2 可知，起爆后约 2.956s 烟幕才进入完整圆柱的有效视线走廊，有效遮蔽持续 1.391643s。该结果为后续优化提供了统一基准。

六、问题二：单机单弹连续参数优化

6.1 决策变量与约束

为避免投放时刻和延迟时间之间的冗余，将决策向量写为 $x=(\theta,v,t_b,\tau)$ ，并由 $t_d=t_b-\tau$ 恢复投放时刻。可行域满足 $0\leq\theta<2\pi$ 、 $70\leq v\leq 140$ 、 $t_b\geq 0$ 、 $0\leq\tau\leq t_b$ ，同时要求起爆高度非负。目标函数为单枚烟幕对 M1 的有效遮蔽时长 $T_1(x)$ 。

$$\max_x T_1(x), s.t. 0\leq\theta<2\pi, 70\leq v\leq 140, t_b\geq 0, 0\leq\tau\leq t_b, z_B\geq 0 \quad (6-1)$$

6.2 分层求解方法

首先在四维可行域内采用差分进化进行粗搜索。多轮搜索均表明较优方案集中于速度上界 $v=140\text{m/s}$ 和延迟下界 $\tau=0$ 附近。该现象具有明确的几何解释：提高无人机速度可扩大起爆点的水平可达范围；在本题允许零延迟时，立即起爆避免自由落体造成的高度损失，使烟幕更早进入导弹—目标视线走廊。由于该方案同时落在速度上界与延迟下界，后文第 10.3 节将进一步通过边界参数敏感性实验检验其数值稳健性。

据此固定 $v=140\text{m/s}$ 、 $\tau=0$ ，将问题降为航向角和起爆时刻的低维优化。对给定航向角，起爆时刻取在完全遮蔽开始的临界事件附近；随后在粗搜索确定的航向邻域内进行一维有界连续精化。最终所有候选均调用统一连续事件模型重新计算。

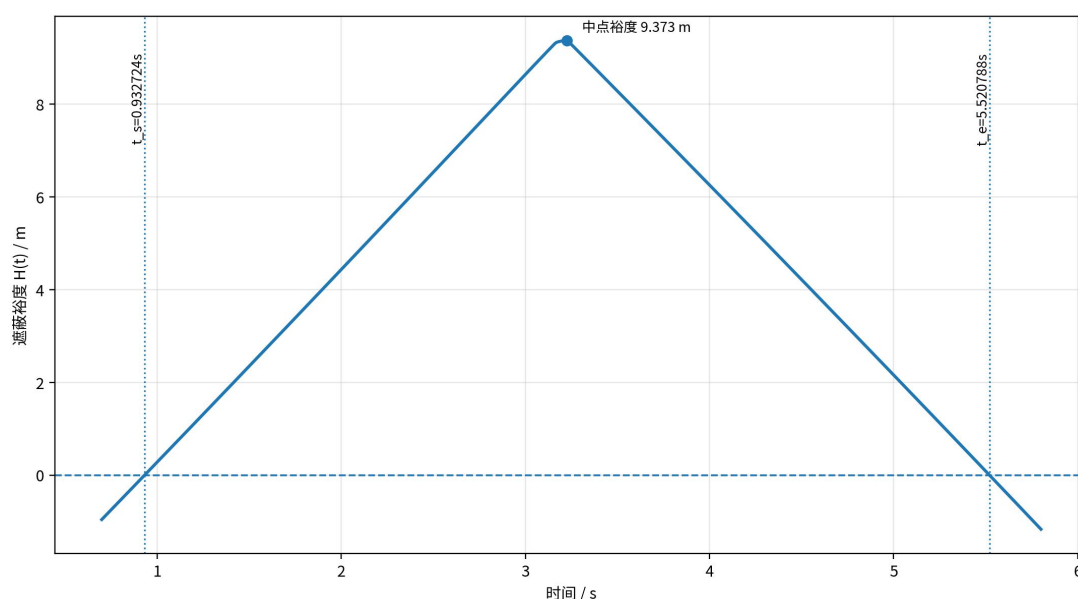


图 5 问题二优化策略下的连续遮蔽裕度变化

图 5 中裕度曲线与零轴的两个交点分别对应烟幕起效和失效时刻。曲线在有效区间内部显著高于零，说明该区间并非由单个时间网格点偶然形成。

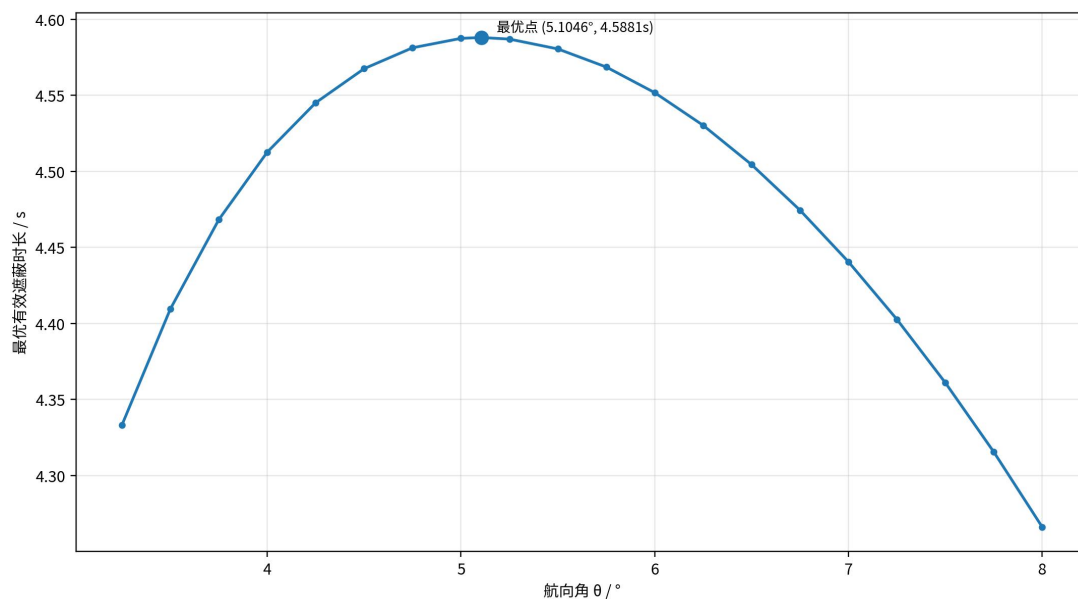


图 6 航向角变化对单弹有效遮蔽时长的影响

如图 6 所示，最优航向位于 5.1° 附近。航向偏差会同时改变烟幕的横向位置和视线切入角度，故目标函数在该邻域形成明显峰值。

6.3 优化结果

表 3 问题二单机单弹优化结果

变量或指标	最优值
航向角 θ	5.104609853°
飞行速度 v	140.000000m/s
投放时刻 t_d	0.932724164s
起爆时刻 t_b	0.932724164s
延迟 τ	0s
投放点/起爆点	$(17930.063485, 11.618405, 1800.000000)\text{m}$
有效遮蔽区间	$[0.932724164, 5.520787703]\text{s}$
有效遮蔽时长	4.588063540s

问题二结果较问题一增加 3.196421s ，有效遮蔽时长达到问题一的约 3.30 倍。改进主要来自两个方面：一是无人机以速度上界快速将烟幕送入 M1 的近端视线走廊；二是零延迟起爆使烟幕从投放时刻开始参与遮蔽，显著提前起效时刻。该结果位于题目允许的速度和延迟边界，因此实际应用中若另设最小引信延迟，需重新求解。

七、问题三：同机三弹临界事件接力

7.1 区间并集目标

三枚烟幕弹共享 FY1 的航向角 θ 和速度 v ，但投放时刻与起爆延迟可分别选择。设第 n 枚烟幕弹按照本文“单云独立完整遮蔽”口径形成的有效区间集合为 I_n ，则问题三目标为最大化 $\mu(I_1 \cup I_2 \cup I_3)$ ，并满足 $t_{n+1}^d - t_n^d \geq 1s$ 。若直接最大化三枚烟幕的单独时长之和，会重复计算时间重叠区间；本文同时不把多个单云的局部空间覆盖进行拼接，因此所得结果具有保守可行性。由于重叠区间不能重复计入目标函数，而过大的时间空窗又会削弱连续防护能力，搜索过程中优先保留重叠较小、空窗较短的候选组合。数值搜索进一步显示，高质量候选逐渐呈现相邻区间首尾接近的结构，据此将临界接力关系作为后续连续局部精修的结构化约束，而不把“严格首尾相接”视为一般情形下的解析必然结论。

$$\max \mu(I_1 \cup I_2 \cup I_3), \quad t_{n+1}^d - t_n^d \geq 1, \quad n = 1, 2 \quad (7-1)$$

7.2 候选生成与临界接力

搜索阶段先在航向角与速度可行域中进行分层采样，并为不同起爆时刻生成单弹候选库；随后枚举满足同航迹和投放间隔约束的三弹组合。全局候选搜索稳定识别出 $\theta \approx \pi$ 、 $v \approx 140m/s$ 的高价值解域后，以相邻投放时刻及首枚引信延迟等变量为连续参数，并利用数值搜索中呈现的“前一枚失效—后一枚起效”近临界结构重构后续起爆时刻，再进行连续局部精修。该结构的作用是减少无效重叠和时间空窗，并非预先假定所有最优解都必须严格满足端点相等。最终所有候选均返回统一连续几何模型复核。

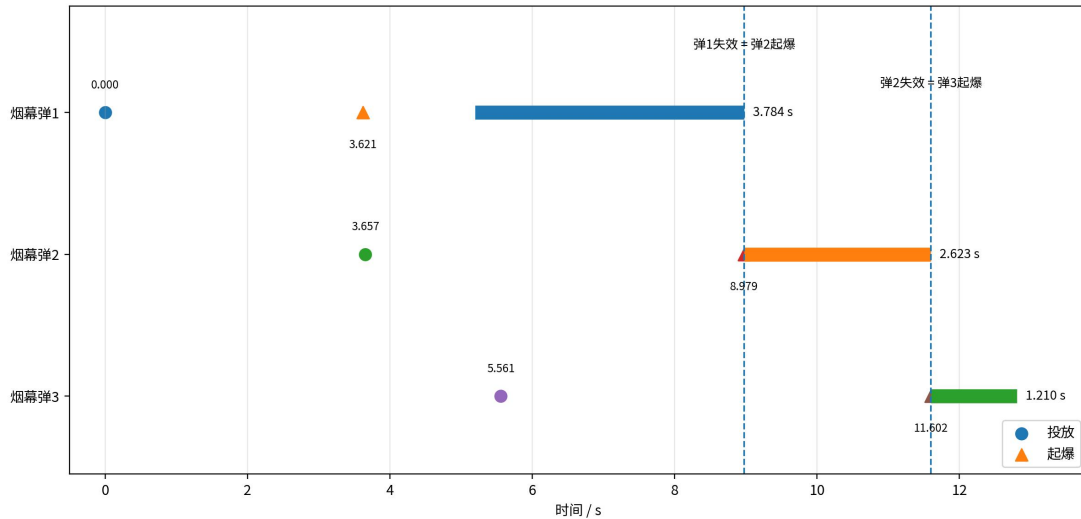


图 7 问题三三枚烟幕弹的临界事件接力关系

图 7 表明三枚烟幕的有效区间首尾相接，既避免了无效空窗，也未形成大面积重叠。后续烟幕的单弹有效时长逐渐缩短，原因在于 M1 持续接近假目标，可利用的几何遮蔽走廊不断压缩。

7.3 三弹策略与结果

表 4FY1 三枚烟幕弹的优化投放与遮蔽结果

弹号	投放时刻/s	起爆时刻/s	有效遮蔽区间/s	时长/s
1	0.000000	3.621061	[5.194503,8.978732]	3.784229
2	3.657263	8.978732	[8.978732,11.602008]	2.623276
3	5.560652	11.602008	[11.602008,12.812051]	1.210043

表 5FY1 三枚烟幕弹的投放点与起爆点

弹号	投放点 D/m	起爆点 B/m
1	(17800.000000,0.000000,1800.000000)	(17293.061116,3.119748,1735.750809)
2	(17287.992877,3.150939,1800.000000)	(16543.001333,7.735685,1661.241645)
3	(17021.523503,4.790815,1800.000000)	(16175.749600,9.995785,1621.159853)

三枚烟幕弹采用共同航向角 179.647401° 和速度 140m/s 。相邻投放间隔分别为 3.657263s 和 1.903389s ，均满足不小于 1s 的约束。三个区间合并后得到连续遮蔽区间 $[5.194503073,12.812051277]\text{s}$ ，总有效时长为 7.617548204s 。与问题二相比，三弹方案将总时长提高 3.029485s ，同时形成长达 7.617548s 的无间断遮蔽。

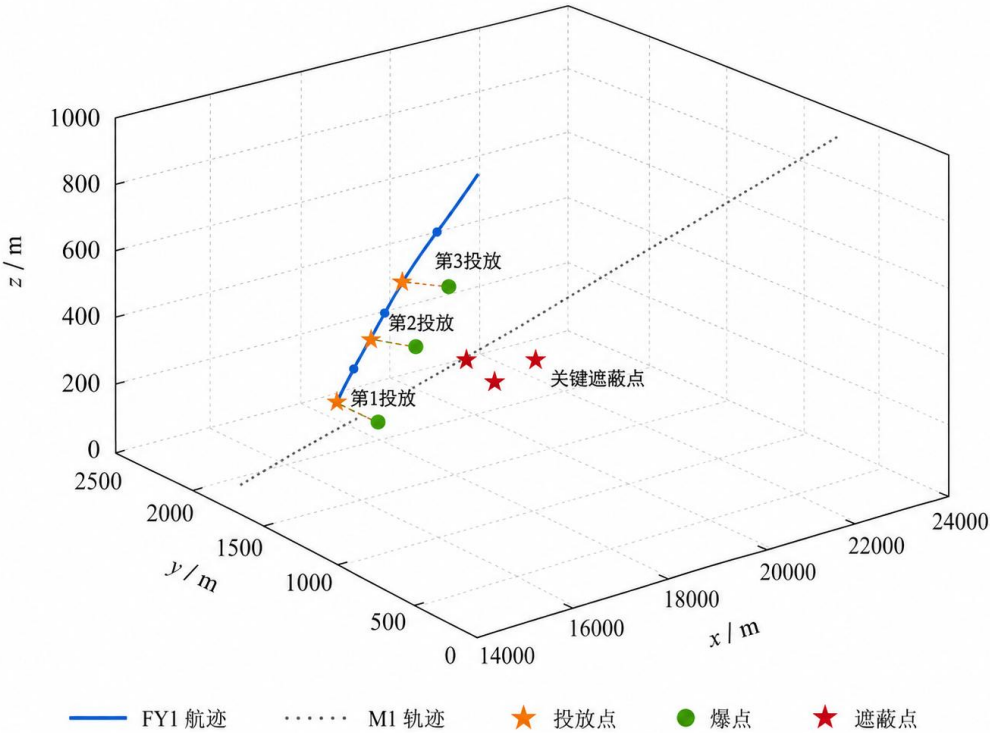


图 8 单机三弹共享航迹下的投放点、起爆点与烟幕有效空间位置

图 8 给出了 FY1 共享航迹上的三个投放点、起爆点及起爆后烟幕下沉位置。三枚烟幕弹沿同一无人机航迹在不同时间形成依次变化的起爆位置，与图 7 所示的时间接力结构相互对应。

八、问题四：三机单弹可达集分解

8.1 单机上界与联合上界

FY1、FY2 和 FY3 之间没有共享航迹或投放资源约束。记第 k 架无人机在其可达策略集内能够达到的理论最大遮蔽时长为 T_k^* ，则任意三机联合方案均满足式（8-1）。本文分别对三架无人机采用与问题二一致的“全局粗搜索—连续几何回代—局部精修”单弹框架，只改变各自初始位置 F_k^0 ，从而得到三组当前数值最优候选。

$$T_{\text{union}} \leq \sum_{k=1}^3 T_k \leq \sum_{k=1}^3 T_k^* \quad (8-1)$$

若三组单机候选分别达到各自的理论全局最优，且对应有效区间两两不交，则三机联合时长等于 $\sum_k T_k^*$ ，从而达到式（8-1）的理论上界。由于本文采用数值全局搜索与局部精修而非解析全局最优证明，后文只主张当前三组单机候选的有效区间互不重叠、其联合时长等于当前三组候选时长之和，不将此等同于严格全局最优证明。

8.2 求解结果

表 6 三架无人机各投一枚烟幕弹的当前数值优化结果

无人机	航向角/(°)	速度/(m/s)	投放时刻/s	延迟/s	有效遮蔽区间/s	时长/s
FY1	5.104610	140.000000	0.932724	0	[0.932724,5.520788]	4.588064
FY2	312.307782	140.000000	9.505982	3.739961	[13.247043,17.236637]	3.989594
FY3	73.255814	139.055742	23.205019	0	[23.235928,26.399497]	3.163569

表 7 三架无人机烟幕弹的投放点与起爆点

无人机	投放点 D/m	起爆点 B/m
FY1	(17930.063485,11.618405,1800.000000)	(17930.063485,11.618405,1800.000000)
FY2	(12895.803962,415.792887,1400.000000)	(13248.242193,28.574000,1331.462207)
FY3	(6929.635613,89.977122,700.000000)	(6929.635613,89.977122,700.000000)

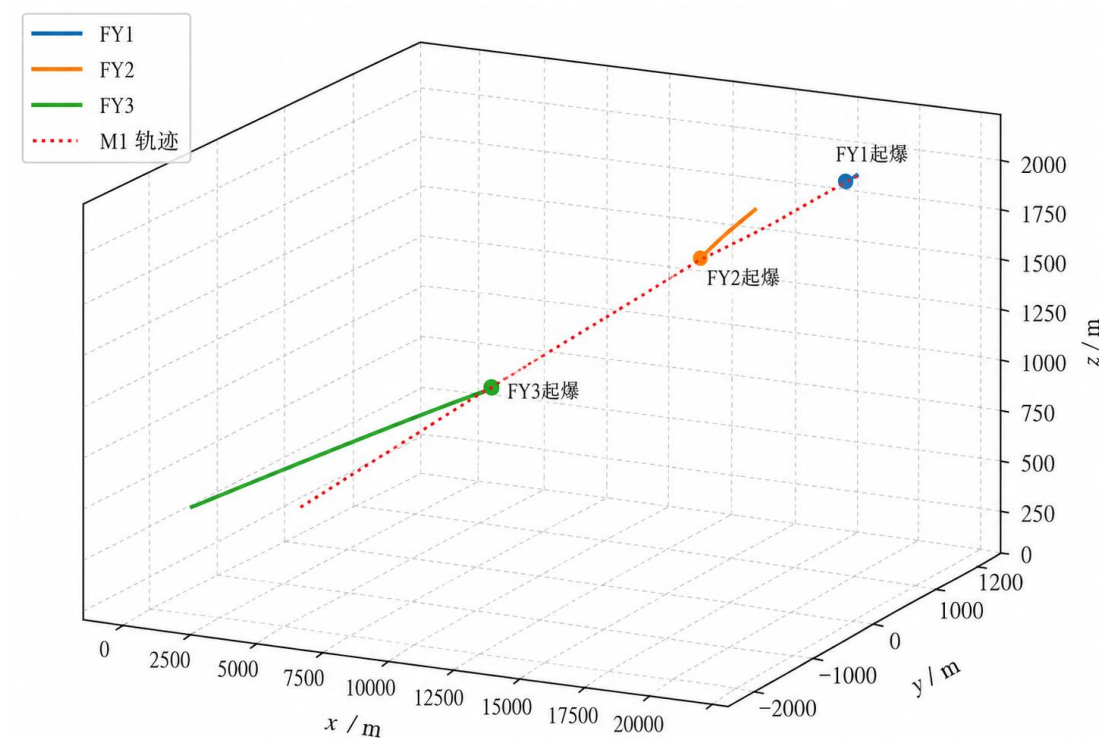


图 9 多无人机独立可达域下的空间协同投放策略

图 9 展示 FY1、FY2 和 FY3 从不同初始位置进入 M1 有效视线区域的空间路径。三架无人机初始纵向位置、横向偏移和飞行高度不同，因此其高价值起爆区域具有不同的时空可达条件，这与后续早、中、晚三个遮蔽时间窗的分工相对应。

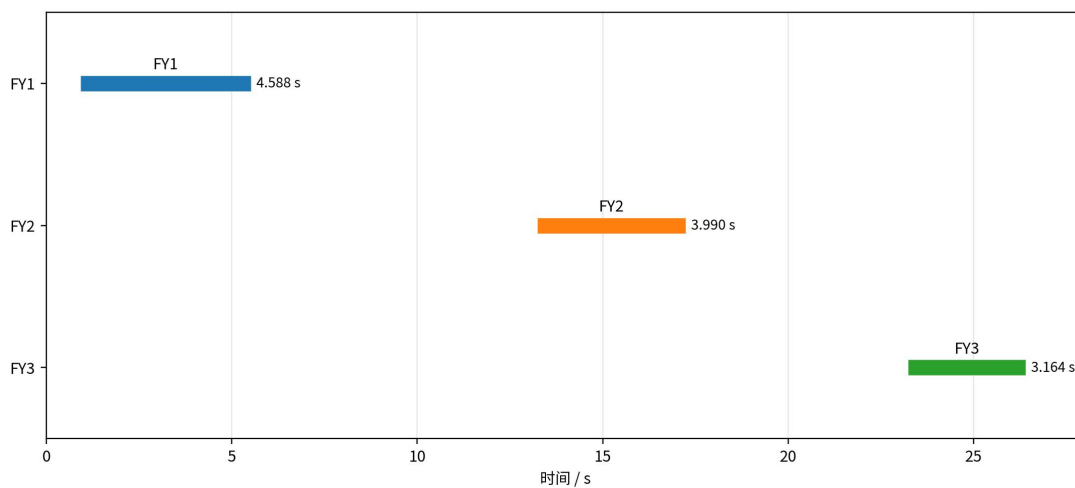


图 10 三架无人机单弹最优遮蔽区间及其并集

表 6 和图 10 显示，三组当前数值优化候选的有效区间之间不存在重叠，因此联合有效时长等于 $4.588063537 + 3.989593987 + 3.163569085 = 11.741226609s$ ，即等于当前三组单机候选时长之和。若三组候选分别达到其严格单机全局最优，则该联合方案同时达到理论上界；本文不以现有数值搜索结果替代该全局最优性证明。三组区间的时间分离与三架无人机不同的初始纵向位置、横向偏移和

飞行高度共同决定的时空可达区域有关：FY1 初始位置已较接近 M1 的有效视线走廊，而 FY2 和 FY3 需要更长时间调整水平位置并进入高价值起爆区域，因此其高质量单弹区间分别落在早、中、晚不同时间段。

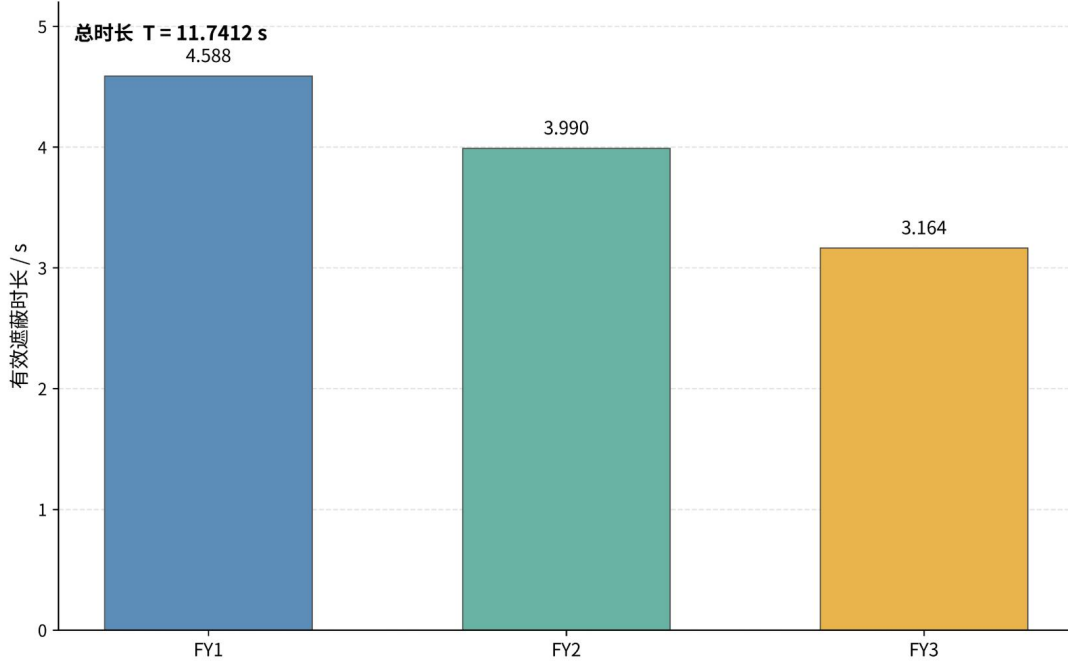


图 11 三架无人机对 M1 有效遮蔽时长的贡献

图 11 表明 FY1、FY2 和 FY3 分别承担早期、中期和后期遮蔽任务。随导弹接近目标，后期可利用的视线走廊缩短，单弹贡献呈递减趋势。

九、问题五：五机三导弹跨目标协同优化

9.1 优化目标与决策结构

问题五中，每架无人机最多投放三枚烟幕弹，同一无人机的全部烟幕弹必须共享航向与速度，但不同烟幕弹可选择不同导弹作为主干扰对象。决策变量同时包含连续变量 $(\theta_k, v_k, t^d_{kn}, t^b_{kn})$ 和离散分配变量 y_{kni} 。令 $y_{kni}=1$ 表示第 k 架无人机第 n 枚烟幕弹以 M_i 为主干扰对象。为与 result3.xlsx 中每枚烟幕弹对应一个“干扰的导弹编号”的任务描述保持一致，本文规定每枚实际使用的烟幕弹至多分配给一枚主目标；其可能对其他导弹产生的附带遮蔽收益不重复计入。从组合优化结构看，本题与经典武器—目标分配（Weapon-Target Assignment, WTA）中的“有限资源—多目标分配”结构具有相似性，但本题资源为烟幕干扰弹、收益为遮蔽时长，同时还包含无人机连续轨迹与投弹时序变量，因此更准确地视为带连续轨迹约束的多资源—多目标混合分配问题；相关资源—目标分配建模可参考文献[7]。

$$\max J = \sum_{i=1}^3 \mu \left(\bigcup_{k,n:y_{kni}=1} I_{kni} \right) \quad (9-1)$$

$$\sum_{i=1}^3 y_{kni} \leq 1, \quad y_{kni} \in \{0,1\}, \quad t^d_{k,n+1} - t^d_{kn} \geq 1 \quad (9-2)$$

主目标 J 鼓励提高三枚导弹的累计受干扰时间，但可能产生导弹间不均衡。为避免只保护某一方向，本文同时计算 $\min_iT_i$ 和三枚导弹同时受遮蔽时长，作为方案评价指标而非额外加权目标。

9.2 分层候选库与组合搜索

第一层针对每个“无人机—导弹”组合，在航向、速度、起爆时刻和延迟的离散网格上生成候选烟幕。为降低中心线近似过于宽松产生的假阳性，搜索代理使用略小于真实半径 10m 的启发式代理半径 R_p ；程序中取 $R_p=8.6m$ 。该数值仅用于候选筛选，不作为物理参数，也不参与正式遮蔽判定。第二层在同一无人机内组合 1~3 枚候选，筛除航向速度不一致、投放间隔不足和起爆高度不合法的计划。第三层联合五架无人机计划，以各导弹时间区间并集增益为评价搜索跨导弹任务分配。第四层围绕入选联合方案，对航向、速度与投弹时序进行连续局部精修；第五层再逐枚调用动态视轮廓模型和连续事件检测，重新计算所有正式区间及目标值。

该分层过程显著减少了高代价几何复核次数，同时保证最终输出满足物理约束。需要指出，候选库离散和组合剪枝会限制搜索范围，因此本节结果表述为高质量可行方案，而非严格全局最优解。

9.3 无人机航迹与任务分配

表 8 问题五五架无人机的航迹及任务分配

无人机	航向角/(°)	速度/(m/s)	实际用弹数	任务序列
FY1	179.647401	140.000000	3	M1→M1→M1
FY2	265.000000	100.000000	3	M2→M1→M3
FY3	87.988611	136.905419	3	M3→M1→M2
FY4	299.500073	136.994415	2	M2→M1
FY5	117.082989	140.000000	1	M3

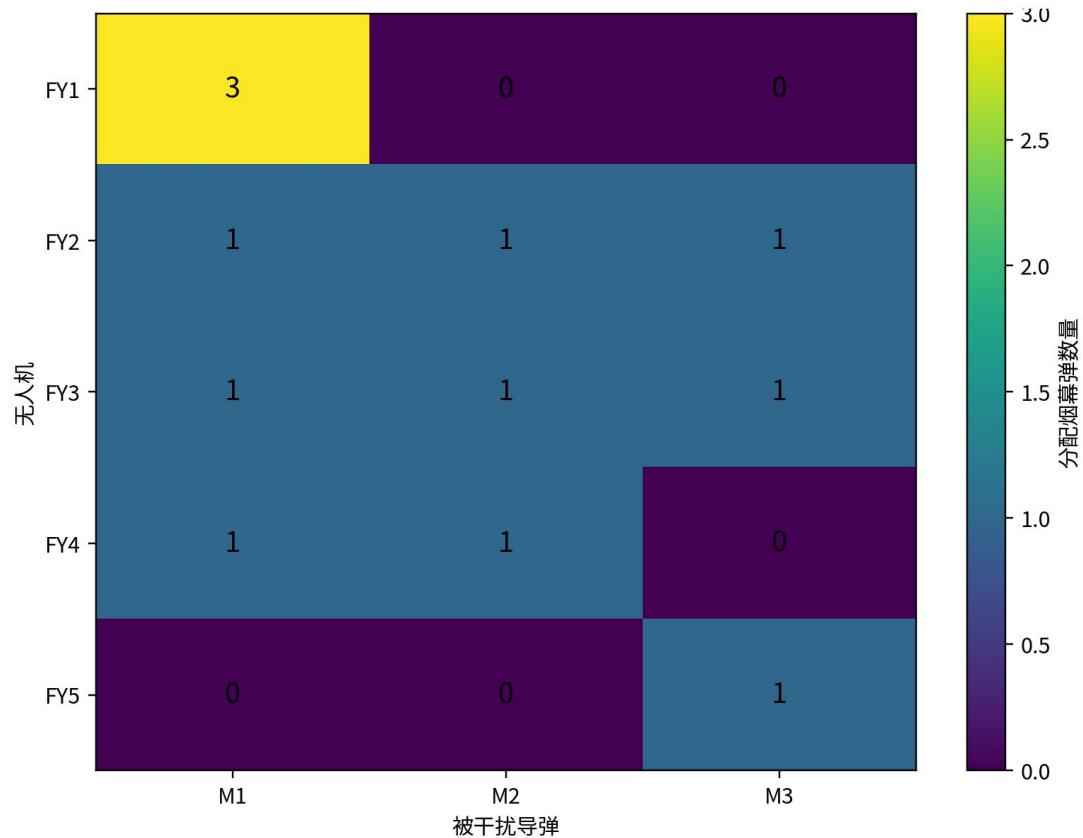


图 12 五架无人机对三枚导弹的烟幕任务分配矩阵

由表 8 和图 12 可知，FY1 专门承担 M1 早期的连续接力；FY2 和 FY3 分别跨三个目标分配烟幕，利用同一航迹在不同时刻穿越不同导弹的有效视线走廊；FY4 补充 M2 与 M1，FY5 专门增强 M3。该结果说明允许同机跨目标分配可提高有限烟幕资源的时空利用率。

9.4 具体投放时序与结果

表 9 问题五 12 枚烟幕弹的投放、起爆与有效遮蔽区间

烟幕弹	导弹	投放时刻/s	起爆时刻/s	有效遮蔽区间/s	时长/s
FY1-1	M1	0.000000	3.621061	[5.194503,8.978732]	3.784229
FY1-2	M1	3.657263	8.978732	[8.978732,11.602008]	2.623276
FY1-3	M1	5.560652	11.602008	[11.602008,12.812051]	1.210043
FY2-1	M2	6.500000	10.000000	[15.588294,19.321694]	3.733400
FY2-2	M1	8.000000	14.000000	[24.305597,27.234782]	2.929185
FY2-3	M3	12.500000	18.000000	[18.289478,20.675954]	2.386476
FY3-1	M3	17.975700	21.100000	[21.583337,24.563887]	2.980550
FY3-2	M1	19.157200	22.500000	[31.901253,34.769416]	2.868162
FY3-3	M2	22.244300	24.000000	[24.768779,27.744913]	2.976134
FY4-1	M2	3.795000	13.500000	[18.932622,22.549516]	3.616894
FY4-2	M1	5.602900	16.500000	[16.615663,20.434584]	3.818922

烟幕弹	导弹	投放时刻/s	起爆时刻/s	有效遮蔽区间/s	时长/s
FY5-1	M3	11.651938	12.949835	[13.146696,16.884682]	3.737986

表 10 问题五 12 枚烟幕弹的投放点与起爆点坐标

烟幕弹	导弹	投放点 D/m	起爆点 B/m
FY1-1	M1	(17800.000,0.000,1800.000)	(17293.061,3.120,1735.751)
FY1-2	M1	(17287.993,3.151,1800.000)	(16543.001,7.736,1661.242)
FY1-3	M1	(17021.524,4.791,1800.000)	(16175.750,9.996,1621.160)
FY2-1	M2	(11943.349,752.473,1400.000)	(11912.844,403.805,1339.975)
FY2-2	M1	(11930.275,603.044,1400.000)	(11877.982,5.327,1223.600)
FY2-3	M3	(11891.055,154.757,1400.000)	(11843.120,-393.150,1251.775)
FY3-1	M3	(6086.376,-540.546,700.000)	(6101.388,-113.075,652.170)
FY3-2	M1	(6092.053,-378.891,700.000)	(6108.115,78.474,645.246)
FY3-3	M2	(6106.887,43.489,700.000)	(6115.323,283.706,684.896)
FY4-1	M2	(11256.009,1547.508,1800.000)	(11910.702,390.344,1338.484)
FY4-2	M1	(11377.968,1331.945,1800.000)	(12113.081,32.643,1218.141)
FY5-1	M3	(12257.314,-547.601,1300.000)	(12174.587,-385.820,1291.746)

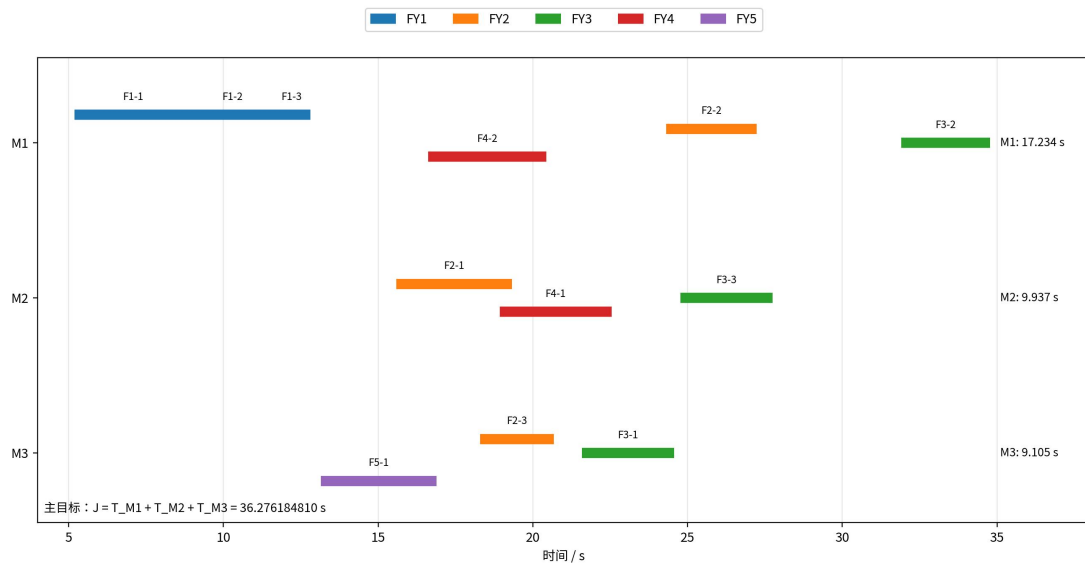


图 13 五架无人机对三枚导弹的多烟幕时域协同覆盖

图 13 直观显示了不同无人机烟幕在三条导弹时间轴上的分布。FY1 形成 M1 的早期连续区间，FY4 和 FY2 在中后期补充 M1；M2 主要由 FY2、FY4 和 FY3 接力；M3 则由 FY5、FY2 和 FY3 分阶段承担。

表 11 三枚导弹的合并遮蔽区间与累计时长

导弹	合并后的有效遮蔽区间/s	累计时长/s
M1	[5.194503,12.812051]; [16.615663,20.434584]; [24.305597,27.234782]; [31.901253,34.769416]	17.233817388
M2	[15.588294,22.549516]; [24.768779,27.744913]	9.937356387
M3	[13.146696,16.884682]; [18.289478,20.675954]; [21.583337,24.563887]	9.105011035

由表 11 可得主目标 J=36.276184810s，最小单导弹累计遮蔽时长为 9.105011035s。三枚导弹同时被有效遮蔽的时间约为 2.414126s，至少一枚导弹被遮蔽的总时间约为 25.083928s。上述四项指标分别反映总体累计干扰收益、最弱方向保护程度、多导弹同步防护能力以及总体作战时域覆盖范围。表 10 中的坐标保留到毫米量级，完整数值以 result3.xlsx 为准。

十、模型检验与结果讨论

10.1 决策自由度增加的性能变化

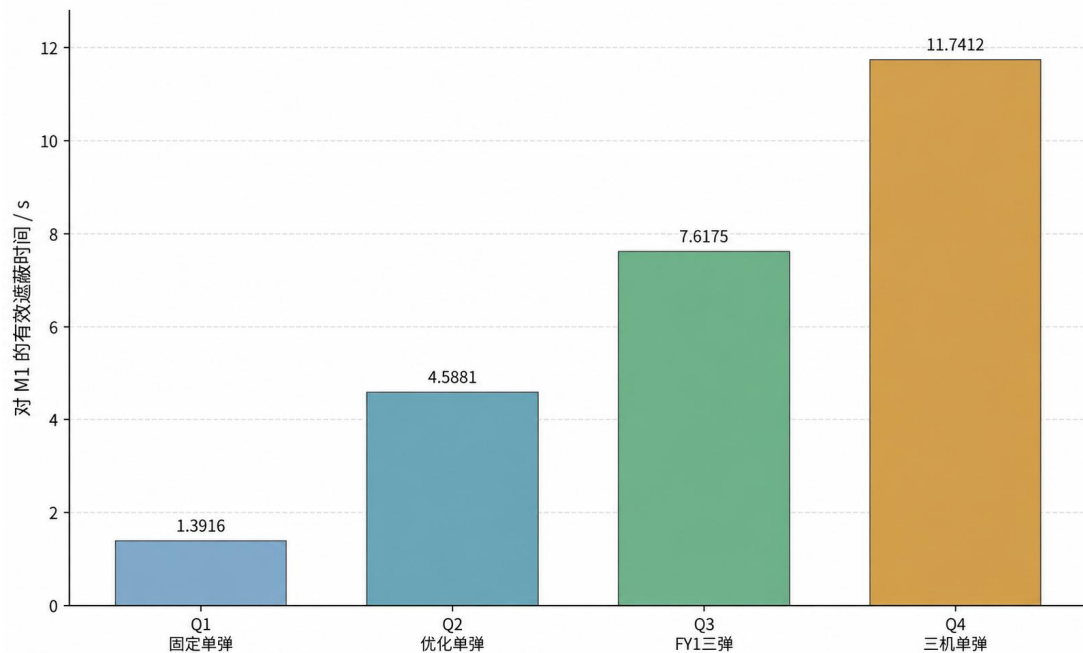


图 14 不同决策自由度下 M1 有效遮蔽性能的逐级提升

图 14 显示，从固定单弹到优化单弹、同机三弹和三机单弹，有效遮蔽时长依次为 1.391643s、4.588064s、7.617548s 和 11.741227s。结果随决策自由度增加而单调提高，且每一步提升均可由新增的空间可达范围或时间接力能力解释，符合模型机制。

10.2 数值分辨率与连续求根收敛检验

为分别检验轮廓离散精度与时间扫描步长对结果的影响，以问题二最优策略为例进行单变量控制：先固定时间扫描步长 $\Delta t=0.03s$ 、逐步提高轮廓离散点数 N ；再固定 $N=96$ 、逐步减小 Δt 。所有设置在粗扫描后均使用 Brent 方法重新确定事件端点。

表 12 问题二数值分辨率的单因素收敛检验

轮廓离散点数 N	时间步长 $\Delta t/s$	连续复核时长/s
固定 $\Delta t=0.03s$		
24	0.03	4.588064
48	0.03	4.588064
72	0.03	4.588064
96	0.03	4.588064
144	0.03	4.588064
固定 $N=96$		
96	0.12	4.588064
96	0.08	4.588064
96	0.05	4.588064
96	0.03	4.588064
96	0.02	4.588064

表 12 表明，在保证异号区间不被漏检的前提下，轮廓离散点数 N 与时间扫描步长 Δt 对正式结果均无实质影响：两个单变量实验中连续复核时长在小数点后 6 位完全一致。最终端点精度由 Brent 连续求根而非离散网格控制，故可适当降低搜索层分辨率而不改变正式结果。

10.3 边界参数的敏感性检验

问题二的数值最优解位于 $\tau=0$ 的约束边界。为检验“允许立即起爆”这一理想化条件是否导致结果失稳，固定 $v=140m/s$ ，并分别规定最小引信延迟 $\tau_{\min}=0、0.05、0.10、0.20s$ ，在原最优邻域内重新优化航向角与起爆时刻，再调用连续几何模型复核。

表 13 最小引信延迟对问题二有效遮蔽时长的影响

最小引信延迟 $\tau_{\min}/s$	连续复核最优时长/s	相对 $\tau_{\min}=0$ 下降/%
0.00	4.588064	0
0.05	4.588062	0.000027
0.10	4.588059	0.000107
0.20	4.588044	0.000434

结果表明，即使人为加入 0.20s 的最小引信延迟，最优有效遮蔽时长的相对下降仍不足 0.001%。因此， $\tau=0$ 虽为数学边界解，但其附近存在较宽的高质量可行域，结论并非依赖单一理想化时刻。

进一步检验速度上界边界的敏感性。固定零延迟，在 $70\sim 140\text{m/s}$ 内以 10m/s 为步长变化飞行速度，并在各速度下重新优化航向角与起爆时刻、再调用连续几何模型复核。结果表明，最优有效遮蔽时长随速度单调递增，由 $v=70\text{m/s}$ 时的 4.576919s 增至 $v=140\text{m/s}$ 时的 4.588064s ，全程变化幅度不足 0.25% 。这说明 $v=140\text{m/s}$ 虽为边界最优解，但目标函数在速度方向上近乎平坦，结果对速度取值不敏感，所得策略并不依赖单一速度边界。

10.4 搜索收敛性检验

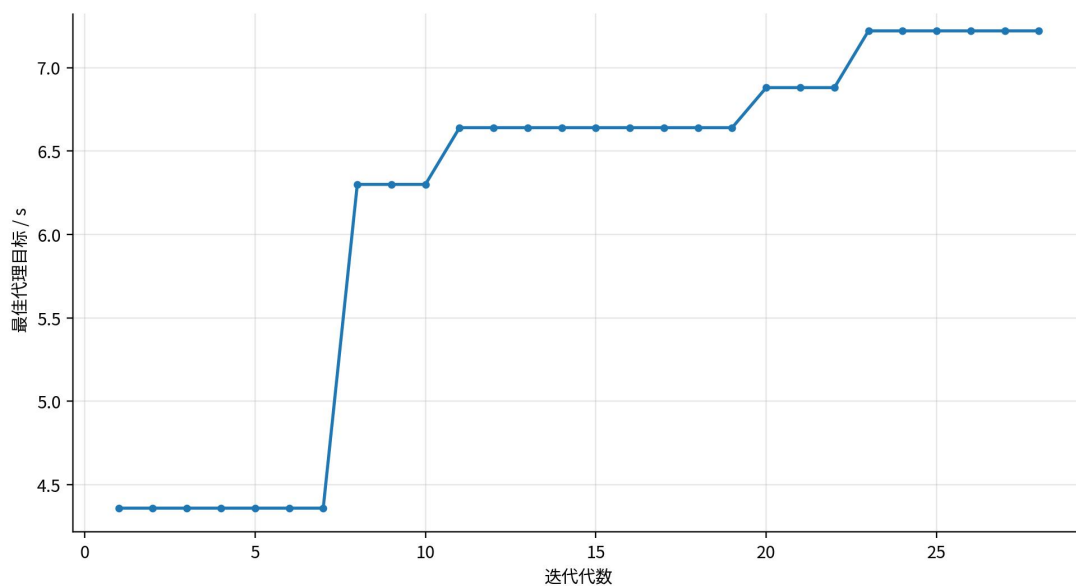


图 15 问题三搜索阶段代理目标的收敛过程

问题三代理目标经历多次阶梯式提升，并在后期稳定于高值区域，如图 15 所示。阶梯结构来自候选组合和接力关系的离散变化，而非连续模型本身不收敛。

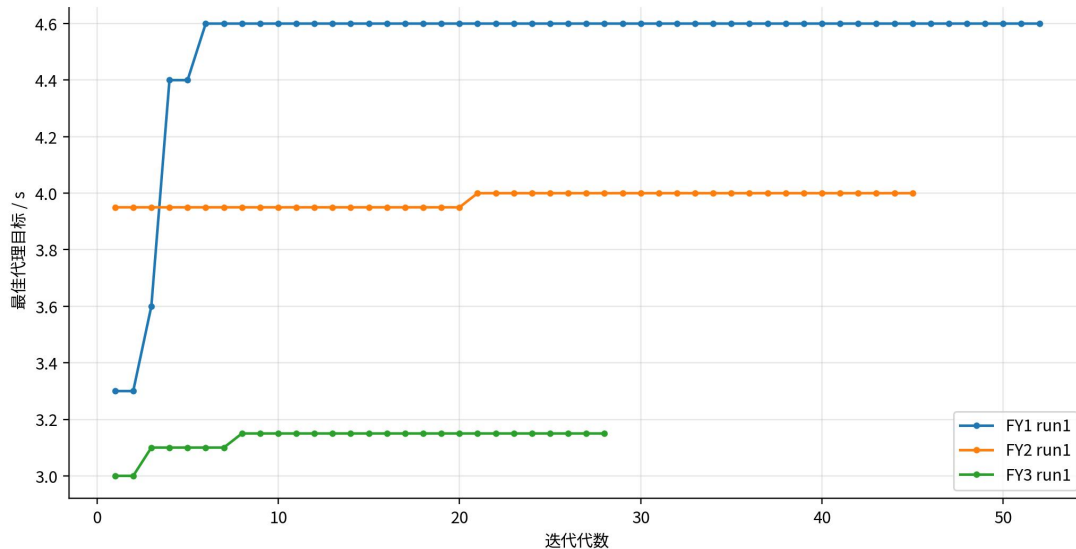


图 16 问题四各单机搜索的代理目标收敛过程

图 16 中三架无人机均在有限迭代内达到稳定平台。其中 FY1 较快达到 4.6s 附近，FY2 和 FY3 的稳定值分别约为 4.0s 和 3.16s，与正式连续复核结果一致。应注意，图中曲线用于说明搜索过程稳定，不构成全局最优性的充分证明。

10.5 完整圆柱高密度几何复核

优化内层以动态边界控制点降低计算量。为检验该近似是否遗漏圆柱侧面、顶面或底面的更不利点，对最终策略的每个有效区间分别选取起点内侧 $t_s + \epsilon$ 、区间中点和终点内侧 $t_e - \epsilon$ 三个关键状态，其中 $\epsilon = 10^{-4}s$ ；每个状态在完整圆柱侧面及上下端面共构造 74,160 个高密度边界采样点，并计算烟幕球心到所有导弹—目标有限线段的最大距离。

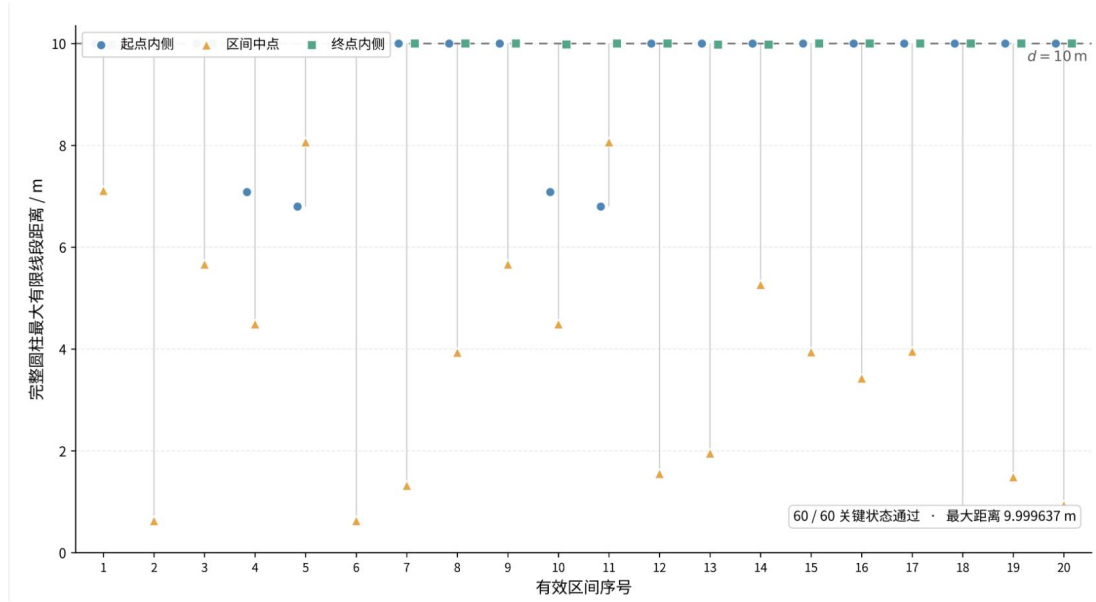


图 17 最终策略在完整圆柱高密度离散条件下的几何遮蔽复核

共检验 60 个关键几何状态，全部满足最大距离小于 10m，最大值为 9.999637m，如图 17 所示。起点内侧和终点内侧的距离接近阈值，符合其邻近连续事件根的性质；区间中点通常具有更大遮蔽余量。该结果为动态边界判定提供数值证据，但不能替代对所有连续时刻和连续表面的解析证明。

10.6 多弹空间联合覆盖口径的讨论

本文正式结果采用“至少一个单云团独立完整遮蔽目标”的保守口径（假设(8)）。若进一步放宽为空间联合覆盖，可定义 $D_{i^{\{multi\}}}(t) = \max_{Q \in \Gamma_i(t)} \min_{k,n} \{d_{ikn}(Q,t)\}$ ：只要目标每个视轮廓点 Q 均被至少一枚烟幕遮蔽，即判为联合有效。由集合包含关系可知，单云独立完整遮蔽成立时，联合覆盖判据必然成立，因此本文问题三至五的正式结果可视为更一般联合覆盖口径下的可行下界。联合口径可能额外计入多个云团分别遮挡目标不同局部区域的时段，其实际增益取决于各云团的相对空间位置与动态视轮廓。

为保证论文数值、result1.xlsx~result3.xlsx 与正式支撑程序之间保持同一判定口径，本文不将尚未纳入正式复现链的联合覆盖辅助结果列入性能比较，也不据此改变问题三至五的最终数值。从最终方案的时域结构看，多枚烟幕的主要协同作用表现为不同时段의接力覆盖，而非同一时刻大量云团对目标不同局部的空间互补，因此保守单云口径与联合覆盖口径之间的差异预计主要集中于少量多云团同时有效的时段。该现象提示本文所得下界在本题参数范围内具有一定紧性，但其精确差距仍需将联合覆盖判据直接纳入优化模型后重新求解才能确定。后续若将联合覆盖作为正式模型，应把 $D_{i^{\{multi\}}}(t)$ 直接纳入连续事件判定与优化目标，并重新求解多弹问题，而不能仅对现有策略作离散后验比较。因而本节的作用是说明保守口径与更一般模型之间的数学关系，并明确本文结果的下界性质。

十一、模型评价与改进

11.1 模型优点

(1) 本模型在优化内层采用圆柱动态控制边界而非目标中心作为遮蔽对象，并在最终策略上使用完整圆柱高密度复核，能够反映有限目标尺度及观察方向变化，避免将“中心被挡”误写为“目标完全被挡”。

(2) 本模型采用导弹—目标有限线段距离，显式排除了烟幕位于导弹后方或目标后方造成的无限直线误判，几何含义清晰。

(3) 本模型把遮蔽判定转化为连续裕度函数的符号判断，并利用事件求根确定区间端点，降低了固定时间步长对有效时长的系统性误差。

(4) 本模型针对问题三至五分别使用临界接力、可达集分解和分层候选分配，充分利用各子问题的结构差异，没有简单地把所有问题交给同一黑箱算法。

(5) 本模型严格区分搜索代理值和正式复核值，并对完整圆柱进行高密度几何审计，使最终结果具有可追溯的数值证据链。

11.2 模型局限

(1) 本模型忽略风场、空气阻力、云团扩散和浓度衰减，仅使用固定半径球体与匀速下沉描述烟幕；在复杂气象条件下，实际云团形态可能明显偏离球形。

(2) 本模型将无人机航向变化视为瞬时完成，未考虑最小转弯半径、加速度和姿态约束，因此结果更适合作为任务层规划方案，而非直接替代飞控层轨迹。

(3) 问题三至五采用候选库和分层剪枝，虽经连续几何复核保证可行，但尚无严格证据证明所得组合达到全局最优。

(4) 动态边界控制点在优化内层降低了计算量，完整圆柱审计只对有限关键状态实施，因此对连续表面的结论仍属于高密度数值验证。

(5) 问题五选择三枚导弹累计遮蔽时长之和作为主目标，体现总收益但不天然保证均衡；若任务更重视最弱方向，应改用最大化 $\min_i T_i$ 或多目标 Pareto 模型。

(6) 多弹问题采用“至少一个单云团独立完全遮蔽目标”的保守判据，未计入多个云团各自遮住目标不同部分时可能形成的空间联合遮蔽，因此问题三至五的有效时长可能低估更一般联合云团模型下的可实现收益。

11.3 改进方向

后续可引入随时间变化的椭球云团或对流—扩散模型，根据风速、风向和烟幕浓度阈值计算有效区域；在无人机运动学中加入最小转弯半径和加速度约束，将直线航迹扩展为 Dubins 曲线或分段最优控制轨迹。对于多烟幕空间联合遮蔽，可采用 $D_i^{\{\text{multi}\}}(t) = \max_{k,n} Q_{\min_{\{k,n\}} d_{\{ikn\}}(Q,t)$ 直接判断多个云团对目标不同局部的联合覆盖，从而消除本文保守单云口径带来的下界效应。针对问

题五，可采用混合整数非线性规划、分支定界与并行差分进化结合的方式扩大搜索覆盖，并同时优化总时长、最小单导弹时长和同时遮蔽时长。对于几何判据，可进一步研究凸目标轮廓极值的解析性质，从理论上证明只需检查有限控制边界。

十二、AI 工具使用声明

本参赛队在竞赛过程中使用人工智能工具进行辅助，主要用于建模方案的可行性讨论与审查、程序代码辅助、论文语言润色以及图表和文档排版。AI 工具仅作为辅助分析与表达工具使用，核心模型选择、数学推导、算法选择、参数确定、程序运行、数值结果及最终结论均由参赛队成员人工审查、修改并确认，详细使用情况见支撑材料《AI 工具使用详情.pdf》。

为保持 AI 工具使用过程的可追溯性，支撑材料中列出所使用工具、主要辅助环节、典型提示方式以及人工采纳与核验情况。参赛队同时查阅了 DeepSeek-V4-Pro 相关官方技术报告[9]作为 AI 工具背景资料；该文献不作为本文数学模型、算法设计或数值结果的直接依据。

十三、参考文献

- [1]CIPOLLAR,GIBLINP.VisualMotionofCurvesandSurfaces[M].Cambridge:CambridgeUniversityPress,2000.
- [2]GIBLINP.Apparentcontours:anoutline[J].PhilosophicalTransactionsoftheRoyalSocietyA,1998,356(1740):1087-1102.
- [3]REDONS,KHEDDARA,COQUILLARTS.Fastcontinuouscollisiondetectionbetweenrigidbodies[J].ComputerGraphicsForum,2002,21(3):279-287.
- [4]BRENTRP.Analgorithmwithguaranteedconvergenceforfindingazeroofafunction[J].TheComputerJournal,1971,14(4):422-425.
- [5]STORNR,PRICEK.Differentialevolution—
Asimpleandefficientheuristicforglobaloptimizationovercontinuouspaces[J].JournalofGlobalOptimization,1997,11(4):341-359.
- [6]MITCHELLIM,BAYENAM,TOMLINJC.Atime-dependentHamilton-
Jacobi formulation of reachable sets for continuous dynamic games[J].IEEE Transactions on Automatic Control,2005,50(7):947-957.
- [7]LIX,WUY,WANGL.Acomprehensivesurveyofweapontargetassignmentproblem:Model,algorithm,andapplication[J].EngineeringApplicationsofArtificialIntelligence,2024,137:109212.
- [8]Anoveleffectivenessevaluationmethodofsmokescreenjammingbasedonthe fusion of directional gradients and local texture features[J].AIP Advances,2022,12(11):115107.
- [9]DEEPSEEK-AI.DeepSeek-V4:TowardsHighlyEfficientMillion-TokenContextIntelligence[R/OL].2026[2026-08-28].https://huggingface.co/deepseek-ai/DeepSeek-V4-Pro/blob/main/DeepSeek_V4.pdf.

附录 A 支撑材料目录

 01_结果文件	2026/8/28 16:34	文件夹
 02_源程序	2026/8/28 16:36	文件夹
 03_验证与中间数据	2026/8/28 16:30	文件夹
 AI工具使用详情.docx	2026/8/28 16:40	DOCX 文档
 requirements.txt	2026/8/28 16:30	文本文档

图 1 支撑材料截图



图 2 支撑材料目录结构

附录 B 五问完整源程序

本附录按公共计算内核、问题一至问题五的顺序给出完整 Python 源程序。各程序与论文采用同一套运动学、动态视轮廓和连续事件检测模型；运行依赖列于附录 B.10。

B.1 公共计算内核：smoke_model_core.py

```
#!/usr/bin/env python
#-*-coding:utf-8-*-
"""Sharedphysics/geometrykernelforthesmoke-screenmodelingproblem.

ThismodulecontainsNOoptimizedfinalstrategies.SolversforQ3-Q5import
thesefunctionsandgeneratestrategiesfromtherawproblemdata.
"""
from__future__importannotations
importmath
fromdataclassesimportdataclass
fromtypingimportDict,Iterable,List,Sequence,Tuple
importnumpyasnp
fromscipy.optimizeimportminimize_scalar,brentq

G_DEFAULT=9.8
MISSILE_SPEED=300.0
SMOKE_R=10.0
SMOKE_SINK=3.0
SMOKE_LIFE=20.0
TARGET_R=7.0
TARGET_H=10.0
TARGET_CENTER=np.array([0.0,200.0,5.0])

MISSILES:Dict[str,np.ndarray]={
    "M1":np.array([2000.0,0.0,2000.0]),
    "M2":np.array([1900.0,600.0,2100.0]),
    "M3":np.array([1800.0,-600.0,1900.0]),
}
DRONES:Dict[str,np.ndarray]={
    "FY1":np.array([17800.0,0.0,1800.0]),
    "FY2":np.array([12000.0,1400.0,1400.0]),
    "FY3":np.array([6000.0,-3000.0,700.0]),
    "FY4":np.array([11000.0,2000.0,1800.0]),
    "FY5":np.array([13000.0,-2000.0,1300.0]),
}

defmissile_velocity(missile:str)->np.ndarray:
    m0=MISSILES[missile]
    returnMISSILE_SPEED*(-m0/np.linalg.norm(m0))

defmissile_end_time(missile:str)->float:
    returnfloat(np.linalg.norm(MISSILES[missile])/MISSILE_SPEED)

defmissile_position(missile:str,t:float|np.ndarray)->np.ndarray:
    returnMISSILES[missile]+np.asarray(t)[...,None]*missile_velocity(missile)

defdirection(theta:float)->np.ndarray:
    returnnp.array([math.cos(theta),math.sin(theta),0.0])

defshot_geometry(drone:str,theta:float,v:float,tb:float,tau:float,
    g:float=G_DEFAULT):
    """Return(td,D,B)foronesmokebomb.

    theta:horizontalheading,+x=0,CCWpositive
    v:UAVspeed
    tb:absolutebursttime
    tau:fusedelayafterdropping
```

```

"""
td=float(tb-tau)
e=direction(theta)
f0=DRONES[drone]
D=f0+v*td*e
B=f0+v*tb*e
B=B.copy()
B[2]-=0.5*g*tau*tau
returntd,D,B

deffeasible_shot(drone:str,theta:float,v:float,tb:float,tau:float,
g:float=G_DEFAULT,tau_min:float=0.0)->bool:
ifnot(70.0<=v<=140.0):
returnFalse
iftb<0.0ortau<tau_min-1e-12ortau>tb+1e-12:
returnFalse
td,_,B=shot_geometry(drone,theta,v,tb,tau,g)
returntd>=-1e-12andB[2]>=-1e-12

defsmoke_center(t:float,drone:str,theta:float,v:float,tb:float,
tau:float,g:float=G_DEFAULT)->np.ndarray:
_,_,B=shot_geometry(drone,theta,v,tb,tau,g)
returnB+np.array([0.0,0.0,-SMOKE_SINK*(t-tb)])

deftarget_rim(a:np.ndarray,z:float)->np.ndarray:
a=np.asarray(a)
returnnp.stack([
TARGET_R*np.cos(a),
200.0+TARGET_R*np.sin(a),
np.full_like(a,z,dtype=float),
],axis=-1)

defdistances_to_segments(P:np.ndarray,A:np.ndarray,B:np.ndarray)->np.ndarray:
AB=B-A
AP=P-A
denom=np.sum(AB*AB,axis=-1)
lam=np.sum(AB*AP,axis=-1)/denom
lam=np.clip(lam,0.0,1.0)
closest=A+lam[... ,None]*AB
returnnp.linalg.norm(P-closest,axis=-1)

defmax_distance_on_rim(t:float,z:float,drone:str,missile:str,
theta:float,v:float,tb:float,tau:float,
coarse_n:int=64,g:float=G_DEFAULT):
M=missile_position(missile,t)
C=smoke_center(t,drone,theta,v,tb,tau,g)
grid=np.linspace(0.0,2*np.pi,coarse_n,endpoint=False)
Q=target_rim(grid,z)
d=distances_to_segments(C,M,Q)
i=int(np.argmax(d))
step=2*np.pi/coarse_n
a0=grid[i]

defobjective(a):
q=target_rim(np.array([a%(2*np.pi)]),z)[0]
return-float(distances_to_segments(C,M,q[None,:])[0])

r=minimize_scalar(
objective,
bounds=(a0-2*step,a0+2*step),
method="bounded",
options={"xatol":1e-11,"maxiter":120},
)
return-float(r.fun),float(r.x%(2*np.pi))

defmargin(t:float,drone:str,missile:str,theta:float,v:float,
tb:float,tau:float,coarse_n:int=64,

```

```

g:float=G_DEFAULT)->float:
ift<tb-1e-12ort>tb+SMOKE_LIFE+1e-12:
return-1e9
d0,_=max_distance_on_rim(t,0.0,drone,missile,theta,v,tb,tau,coarse_n,g)
d1,_=max_distance_on_rim(t,TARGET_H,drone,missile,theta,v,tb,tau,coarse_n,g)
returnSMOKE_R-max(d0,d1)

defexact_intervals(drone:str,missile:str,theta:float,v:float,
tb:float,tau:float,scan_dt:float=0.03,
scan_n:int=40,root_n:int=120,
g:float=G_DEFAULT)->List[Tuple[float,float]]:
ifnotfeasible_shot(drone,theta,v,tb,tau,g):
return[]
te=min(tb+SMOKE_LIFE,missile_end_time(missile))
ifte<=tb:
return[]
ts=np.arange(tb,te+0.5*scan_dt,scan_dt)
vals=np.array([margin(t,drone,missile,theta,v,tb,tau,scan_n,g)fortints])
ok=vals>=0.0
out:List[Tuple[float,float]]=[]
s=tbifok[0]elseNone
foriinrange(len(ts)-1):
if(notok[i])andok[i+1]:
s=brentq(
lambdatt:margin(tt,drone,missile,theta,v,tb,tau,root_n,g),
ts[i],ts[i+1],xtol=1e-10,rtol=1e-12,
)
elifok[i]and(notok[i+1]):
e=brentq(
lambdatt:margin(tt,drone,missile,theta,v,tb,tau,root_n,g),
ts[i],ts[i+1],xtol=1e-10,rtol=1e-12,
)
out.append((sifsisnotNoneelsets[i],e))
s=None
ifsisnotNone:
out.append((s,te))
returnout

defmerge_intervals(intervals:Sequence[Tuple[float,float]]):
iv=sorted((float(a),float(b))fora,binintervalsifb>a)
ifnotiv:
return[],0.0
out=[]
s,e=iv[0]
fora,biniv[1:]:
ifa<=e+1e-10:
e=max(e,b)
else:
out.append((s,e))
s,e=a,b
out.append((s,e))
returnout,float(sum(b-afora,binout))

defexact_duration(drone:str,missile:str,theta:float,v:float,
tb:float,tau:float,**kwargs):
iv=exact_intervals(drone,missile,theta,v,tb,tau,**kwargs)
returnsum(b-afora,biniv),iv

classProxyEvaluator:
"""Fasttime-grid/rim-gridevaluatorusedonlyforcandidategeneration."""
def__init__(self,missile:str,dt:float=0.08,horizon:float|None=None,
rim_n:int=20,effective_radius:float=SMOKE_R):
self.missile=missile
self.dt=float(dt)
self.radius=float(effective_radius)
te=missile_end_time(missile)ifhorizonisNoneelsemin(horizon,missile_end_time(missile))
self.ts=np.arange(0.0,te+1e-12,self.dt)
self.M=missile_position(missile,self.ts)
a=np.linspace(0,2*np.pi,rim_n,endpoint=False)

```

```

qb=np.c_[TARGET_R*np.cos(a),200+TARGET_R*np.sin(a),np.zeros(rim_n)]
qt=qb.copy();qt[:,2]=TARGET_H
self.Q=np.vstack([qb,qt])
self.AB=self.Q[None,:,:]-self.M[:,None,:]
self.den=np.sum(self.AB*self.AB,axis=2)

defmargins(self,drone:str,theta:float,v:float,tb:float,tau:float,
g:float=G_DEFAULT)->np.ndarray:
ifnotfeasible_shot(drone,theta,v,tb,tau,g):
returnnp.full(len(self.ts),-1e9)
_,_,B=shot_geometry(drone,theta,v,tb,tau,g)
C=B[None,:]+np.c_[np.zeros_like(self.ts),np.zeros_like(self.ts),-SMOKE_SINK*(self.ts-tb)]
AP=C-self.M
lam=np.sum(self.AB*AP[:,None,:],axis=2)/self.den
lam=np.clip(lam,0.0,1.0)
closest=self.M[:,None,:]+lam[:,None]*self.AB
d=np.linalg.norm(C[:,None,:]-closest,axis=2).max(axis=1)
returnself.radius-d

definterval_mask(self,drone:str,theta:float,v:float,tb:float,tau:float,
g:float=G_DEFAULT):
mg=self.margins(drone,theta,v,tb,tau,g)
return(mg>=0.0)&(self.ts>=tb)&(self.ts<=tb+SMOKE_LIFE)

defduration(self,drone:str,theta:float,v:float,tb:float,tau:float,
g:float=G_DEFAULT,soft:bool=False,eps:float=1.5):
mg=self.margins(drone,theta,v,tb,tau,g)
active=(self.ts>=tb)&(self.ts<=tb+SMOKE_LIFE)
ifsoft:
s=1.0/(1.0+np.exp(-np.clip(mg/eps,-40,40)))
returnfloat(np.sum(s*active)*self.dt)
returnfloat(np.count_nonzero((mg>=0.0)&active)*self.dt)

defintervals(self,drone:str,theta:float,v:float,tb:float,tau:float,
g:float=G_DEFAULT):
ok=self.interval_mask(drone,theta,v,tb,tau,g)
idx=np.flatnonzero(ok)
iflen(idx)==0:
return[]
groups=np.split(idx,np.where(np.diff(idx)>1)[0]+1)
return[(float(self.ts[g[0]]),float(self.ts[g[-1]]+self.dt))forgingroupsiflen(g)]

defcylinder_boundary_points(n_theta:int=720,n_z:int=61,n_r:int=80)->np.ndarray:
"""Denseboundarypoints:sidesurface+top/bottomdisks.

Thisisforindependentnumericalaudit,notforoptimization.
"""
a=np.linspace(0.0,2*np.pi,n_theta,endpoint=False)
z=np.linspace(0.0,TARGET_H,n_z)
#side
aa,zz=np.meshgrid(a,z,indexing='ij')
side=np.c_[TARGET_R*np.cos(aa.ravel()),200.0+TARGET_R*np.sin(aa.ravel()),zz.ravel()]
#diskswithpolarrings
rr=np.linspace(0.0,TARGET_R,n_r)
ar,rgrid=np.meshgrid(a,rr,indexing='ij')
xy=np.c_[rgrid.ravel()*np.cos(ar.ravel()),200.0+rgrid.ravel()*np.sin(ar.ravel())]
bot=np.c_[xy,np.zeros(len(xy))]
top=np.c_[xy,np.full(len(xy),TARGET_H)]
returnnp.vstack([side,bot,top])

defdefense_cylinder_max_distance(t:float,drone:str,missile:str,
theta:float,v:float,tb:float,tau:float,
points:np.ndarray|None=None,
g:float=G_DEFAULT):
ifpointsisNone:
points=cylinder_boundary_points()
M=missile_position(missile,t)
C=smoke_center(t,drone,theta,v,tb,tau,g)
d=distances_to_segments(C,M,points)
i=int(np.argmax(d))
returnfloat(d[i]),points[i].copy()

```


B.2 问题一：problem1.py

```
#-*-coding:utf-8-*-
"""
2025 数学建模 A 题问题 1
基于“视轮廓+线段-球相交+连续事件检测”的有效遮蔽时长计算

运行：
pythonproblem1_spherical_occlusion.py

依赖：
numpyscipymatplotlib

输出：
1)控制台打印投弹点、起爆点、遮蔽起止时刻与总时长
2)problem1_margin.png 遮蔽裕度曲线

说明：
-t=0 为雷达发现来袭导弹、无人机受领任务时刻。
-真目标是半径 7m、高 10m 的竖直圆柱。
-对圆柱上下边界圆周进行连续角度极值搜索；圆柱侧面投影边界由上下端点连接，
因而上下轮廓圆周控制整个视轮廓是否落入烟幕遮蔽锥。
-对任一目标轮廓点 Q，若线段 M(t)Q 与半径 10m 的烟幕球相交，则该方向被遮挡。
-定义 max_dist(t)为烟幕中心到所有目标轮廓视线段的最大最短距离。
当 max_dist(t)<=10m 时，整个目标视轮廓被遮挡。
"""

importnumpyasnp
importmatplotlib.pyplotasplt
fromscipy.optimizeimportminimize_scalar,brentq

#=====
#1. 常量与题目参数
#=====
G=9.8#重力加速度 m/s^2
MISSILE_SPEED=300.0#导弹速度 m/s
UAV_SPEED=120.0#FY1 速度 m/s
SMOKE_R=10.0#有效烟幕半径 m
SMOKE_SINK=3.0#烟幕下沉速度 m/s
SMOKE_LIFE=20.0#烟幕有效持续时间 s
TARGET_R=7.0#真目标圆柱半径 m
TARGET_H=10.0#真目标圆柱高度 m

M0=np.array([2000.0,0.0,2000.0])
F0=np.array([1780.0,0.0,1800.0])
TARGET_BOTTOM_CENTER=np.array([0.0,200.0,0.0])

DROP_TIME=1.5#受领任务后投放时刻 s
FUSE_DELAY=3.6#投放后到起爆的时间 s
BURST_TIME=DROP_TIME+FUSE_DELAY

#M1 始终直指原点
MISSILE_DIR=-M0/np.linalg.norm(M0)
MISSILE_V=MISSILE_SPEED*MISSILE_DIR

#FY1 等高度朝假目标方向飞行：水平面内沿 x 轴负方向
UAV_V=np.array([-UAV_SPEED,0.0,0.0])

#投弹点
DROP_POINT=F0+UAV_V*DROP_TIME

#起爆点：水平保留无人机速度，竖直自由落体
BURST_POINT=np.array([
    DROP_POINT[0]+UAV_V[0]*FUSE_DELAY,
    DROP_POINT[1],
    DROP_POINT[2]-0.5*G*FUSE_DELAY**2,
])

SMOKE_END_TIME=BURST_TIME+SMOKE_LIFE

#=====
```

```

#2. 运动方程
#=====
def missile_position(t:float)->np.ndarray:
    """M1 在全局时刻 t 的位置。"""
    return M0+MISSILE_V*t

def smoke_center(t:float)->np.ndarray:
    """烟幕起爆后在全局时刻 t 的球心位置。"""
    return BURST_POINT+np.array([0.0,0.0,0,-SMOKE_SINK*(t-BURST_TIME)])

def target_rim(theta:np.ndarray,z:float)->np.ndarray:
    """圆柱目标 z=常数的边界圆周。theta 可为标量或数组。"""
    theta=np.asarray(theta)
    x=TARGET_R*np.cos(theta)
    y=200.0+TARGET_R*np.sin(theta)
    z_arr=np.full_like(theta,z,dtype=float)
    return np.stack([x,y,z_arr],axis=-1)

#=====
#3. 几何判定
#=====
def distances_center_to_segments(P:np.ndarray,
    A:np.ndarray,
    B:np.ndarray)->np.ndarray:
    """
    计算点 P 到一组线段 AB 的最短距离。

    P:(3,)
    A:(3,)——导弹位置
    B:(...,3)——一组目标轮廓点
    返回:(...)距离

    使用线段而不是无限直线，可自动避免把“导弹身后的烟幕”误判为遮蔽。
    """
    AB=B-A
    AP=P-A
    denom=np.sum(AB*AB,axis=-1)
    lam=np.sum(AB*AP,axis=-1)/denom
    lam=np.clip(lam,0.0,1.0)
    closest=A+lam[...,None]*AB
    return np.linalg.norm(P-closest,axis=-1)

def max_distance_on_one_rim(t:float,
    z:float,
    coarse_n:int=720)->tuple[float,float]:
    """
    对 z=0 或 z=10 的圆周，求烟幕球心到“导弹->圆周点”视线段的最大最短距离。

    先均匀粗搜定位最大值，再用一维有界优化精修 theta。
    返回(最大距离,对应 theta)。
    """
    M=missile_position(t)
    C=smoke_center(t)

    #endpoint=False 避免 0 与 2pi 重复
    theta_grid=np.linspace(0.0,2.0*np.pi,coarse_n,endpoint=False)
    Q=target_rim(theta_grid,z)
    d=distances_center_to_segments(C,M,Q)
    i=int(np.argmax(d))

    #在粗最大点左右各扩 2 个网格宽度，局部连续优化
    step=2.0*np.pi/coarse_n
    theta0=theta_grid[i]
    left=theta0-2.0*step
    right=theta0+2.0*step

    def objective(theta:float)->float:
        q=target_rim(np.array([theta%(2.0*np.pi)]),z)[0]
        dist=distances_center_to_segments(C,M,q[None,:])[0]

```

```

return -float(dist)#maximizedistance<=>minimizenegativedistance

result=minimize_scalar(
    objective,
    bounds=(left,right),
    method="bounded",
    options={"xatol":1e-13,"maxiter":200},
)

theta_star=result.x%(2.0*np.pi)
max_dist=-float(result.fun)
return max_dist,theta_star

def max_view_contour_distance(t:float,
    coarse_n:int=720)->tuple[float,str,float]:
    """
    圆柱完整视轮廓上的控制距离。

    检查上下两个端面轮廓圆周，取更不利者。
    返回:
    max_dist,controlling_rim,theta_star
    """
    d_bottom,th_bottom=_max_distance_on_one_rim(t,0.0,coarse_n)
    d_top,th_top=_max_distance_on_one_rim(t,TARGET_H,coarse_n)

    if d_bottom>=d_top:
        return d_bottom,"bottom",th_bottom
    return d_top,"top",th_top

def occlusion_margin(t:float,coarse_n:int=720)->float:
    """
    遮蔽裕度: SMOKE_R-max_view_contour_distance。

    margin>=0: 整个圆柱视轮廓均被烟幕遮挡
    margin<0: 至少存在一个目标轮廓方向未被遮挡
    """
    max_dist,_,_=max_view_contour_distance(t,coarse_n)
    return SMOKE_R-max_dist

#=====
#4. 自动寻找有效遮蔽区间
#=====
def find_occlusion_intervals(scan_dt:float=0.02,
    coarse_n_scan:int=360,
    coarse_n_refine:int=1440):
    """
    先粗扫描定位符号变化，再用 Brent 法连续求根精修。

    返回: [(start,end),...]
    """
    ts=np.arange(BURST_TIME,SMOKE_END_TIME+0.5*scan_dt,scan_dt)
    margins=np.array([occlusion_margin(t,coarse_n_scan) for t in ts])
    valid=margins>=0.0

    intervals=[]
    start=None

    #若起爆瞬间已经有效
    if valid[0]:
        start=BURST_TIME

    for i in range(len(ts)-1):
        if (not valid[i]) and valid[i+1]:
            #进入遮蔽: margin 从负到正
            root=brentq(
                lambda x: occlusion_margin(x,coarse_n_refine),
                ts[i],ts[i+1],
                xtol=1e-11,rtol=1e-13,maxiter=100,
            )
            start=root

```

```

elif valid[i] and (not valid[i+1]):
    #离开遮蔽: margin 从正到负
    root=brentq(
        lambda x:occlusion_margin(x,coarse_n_refine),
        ts[i],ts[i+1],
        xtol=1e-11,rtol=1e-13,maxiter=100,
    )
    if start is None:
        start=ts[i]
    intervals.append((start,root))
    start=None

if start is not None:
    intervals.append((start,SMOKE_END_TIME))

return intervals,ts,margins

#=====
#5. 额外事件: 导弹进入/离开烟幕球
#=====
def missile_smoke_distance(t:float)->float:
    return float(np.linalg.norm(missile_position(t)-smoke_center(t)))

def find_missile_sphere_crossings(scan_dt:float=0.01):
    """求  $|M(t)-C(t)|=10$  的所有根。"""
    ts=np.arange(BURST_TIME,SMOKE_END_TIME+scan_dt/2,scan_dt)
    vals=np.array([missile_smoke_distance(t)-SMOKE_R for t in ts])
    roots=[]
    for i in range(len(ts)-1):
        if vals[i]==0:
            roots.append(ts[i])
        elif vals[i]*vals[i+1]<0:
            roots.append(brentq(
                lambda x:missile_smoke_distance(x)-SMOKE_R,
                ts[i],ts[i+1],xtol=1e-12,rtol=1e-14
            ))
    #去重
    unique=[]
    for r in roots:
        if not unique or abs(r-unique[-1])>1e-8:
            unique.append(r)
    return unique

#=====
#6. 全边界密集采样验证 (非主求解)
#=====
def defense_surface_validation(t:float,
    n_theta:int=1440,
    n_z:int=101,
    n_r:int=80)->float:
    """
    用高密度采样验证整个圆柱边界 (侧面+上下圆盘)。
    返回所有采样目标点对应视线段的最大距离。

    此函数只用于数值核验, 不参与最终根求解。
    """
    M=missile_position(t)
    C=smoke_center(t)
    theta=np.linspace(0.0,2.0*np.pi,n_theta,endpoint=False)

    max_d=0.0

    #侧面: r=7,zin[0,10]
    for z in np.linspace(0.0,TARGET_H,n_z):
        Q=target_rim(theta,z)
        d=distances_center_to_segments(C,M,Q)
        max_d=max(max_d,float(np.max(d)))

    #上下圆盘: rin[0,7]

```

```

forzin(0.0,TARGET_H):
forrinnp.linspace(0.0,TARGET_R,n_r):
Q=np.stack([
r*np.cos(theta),
200.0+r*np.sin(theta),
np.full_like(theta,z),
],axis=-1)
d=distances_center_to_segments(C,M,Q)
max_d=max(max_d,float(np.max(d)))

returnmax_d

#=====
#7.主程序
#=====
defmain():
np.set_printoptions(precision=9,suppress=True)

print("="*66)
print("问题 1: 基于视轮廓与连续事件检测的烟幕有效遮蔽计算")
print("="*66)

print(f"M1 速度向量:{MISSILE_V}")
print(f"投弹时刻 t_d:{DROP_TIME:.6f}s")
print(f"投弹点 D:{DROP_POINT}")
print(f"起爆延迟:{FUSE_DELAY:.6f}s")
print(f"起爆时刻 t_b:{BURST_TIME:.6f}s")
print(f"起爆点 B:{BURST_POINT}")
print(f"烟幕有效区间:[{BURST_TIME:.6f},{SMOKE_END_TIME:.6f}]s")
print()

intervals,ts_scan,margins_scan=find_occlusion_intervals()

ifnotintervals:
print("未发现有效遮蔽区间。")
return

total=sum(b-afora,binintervals)

print("自动检测到的有效遮蔽区间: ")
fork,(a,b)inenumerate(intervals,1):
print(f"区间{k}:[{a:.9f},{b:.9f}]s,时长={b-a:.9f}s")
print(f"\n 总有效遮蔽时长:{total:.9f}s")
print(f"四位小数:{total:.4f}s")

#输出控制轮廓点
t_start=intervals[0][0]
d_start,rim,theta=max_view_contour_distance(t_start,coarse_n=2880)
z=0.0ifrim=="bottom"elseTARGET_H
q_star=target_rim(np.array([theta]),z)[0]
print("\n 遮蔽开始临界事件: ")
print(f"t_start:{t_start:.9f}s")
print(f"控制边界:{rim}")
print(f"theta*:{theta:.9f}rad={np.degrees(theta):.6f}deg")
print(f"临界目标点 Q*:{q_star}")
print(f"最大线段距离:{d_start:.9f}m")

#导弹进入/离开烟幕球
roots=find_missile_sphere_crossings()
ifroots:
print("\nM1 与烟幕球边界的交会时刻: ")
fori,rinenumerate(roots,1):
print(f"root{i}:{r:.9f}s")
iflen(roots)>=2:
print(f"M1 位于烟幕内时长:{roots[1]-roots[0]:.9f}s")

#全圆柱边界高密度检验: 在开始时刻稍后取一个很小 epsilon, 避免根点数值误差
eps=1e-6
validation_t=t_start+eps
dense_max=dense_surface_validation(validation_t)
print("\n 全圆柱边界高密度采样检验: ")
print(f"检验时刻:{validation_t:.9f}s")

```

```

print(f"全边界最大线段距离:{dense_max:.9f}m")
print(f"是否<=10m:{dense_max<=SMOKE_R}")

#绘图时用已算的粗扫描 margin: 额外标注精确根
plt.figure(figsize=(9,5.2))
plt.plot(ts_scan,margins_scan,linewidth=1.7,label="occlusionmargin=10-maxdistance")
plt.axhline(0.0,linewidth=1.0,linestyle="--",label="criticalmargin=0")
for i,(a,b)inenumerate(intervals):
    plt.axvline(a,linewidth=1.0,linestyle=":")
    plt.axvline(b,linewidth=1.0,linestyle=":")
    plt.axvspan(a,b,alpha=0.12,label="effectiveocclusion" if i==0 else None)

plt.xlabel("time/s")
plt.ylabel("occlusionmargin/m")
plt.title("Problem1:continuousocclusionmargin")
plt.grid(alpha=0.25)
plt.legend()
plt.tight_layout()
out_fig="problem1_margin.png"
plt.savefig(out_fig,dpi=220)
print(f"\n 遮蔽裕度曲线已保存为:{out_fig}")

if __name__=="__main__":
    main()

```

B.3 问题二：problem2.py

```
#!/usr/bin/env python
# coding: utf-8
```

数学建模 A 题问题 2

基于“动态视轮廓+连续事件检测+全局搜索/边界精修”的单机单弹最优投放策略

依赖：

numpy, scipy, matplotlib

运行：

python problem2_occlusion_optimization.py

输出：

- 最优航向角、无人机速度、投放/起爆时刻、投放/起爆点

- 精确有效遮蔽区间与总时长

- problem2_margin.png

核心思想：

1) 对任一策略 (θ, v, t_b, τ) ，计算烟幕球心到“导弹→圆柱目标视轮廓”所有视线段的最大距离 $D(t)$ 。

2) $\text{margin}(t) = 10 - D(t) > 0$ 时，认为整个真目标视轮廓被有效遮蔽。

3) 先用低成本的 4 维差分进化搜索确认最优区域；

4) 数值搜索显示最优结构落在 $v = 140\text{m/s}$ 、 $\tau = 0$ 的边界；

5) 在该边界上用连续相切事件 $\text{margin}(t_b) = 0$ 降维，并精确优化航向角。

```
"""
```

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import minimize_scalar, brentq, differential_evolution
```

```
#-----
```

```
#1. 常量
```

```
#-----
```

```
G = 9.8
```

```
MISSILE_SPEED = 300.0
```

```
SMOKE_R = 10.0
```

```
SMOKE_SINK = 3.0
```

```
SMOKE_LIFE = 20.0
```

```
TARGET_R = 7.0
```

```
TARGET_H = 10.0
```

```
M0 = np.array([2000.0, 0.0, 2000.0])
```

```
F0 = np.array([1780.0, 0.0, 1800.0])
```

```
MISSILE_V = MISSILE_SPEED * (-M0 / np.linalg.norm(M0))
```

```
MISSILE_END_TIME = np.linalg.norm(M0) / MISSILE_SPEED
```

```
def missile_position(t):
```

```
    return M0 + MISSILE_V * t
```

```
def decode_strategy(x):
```

```
    """
```

```
    x = (theta, v, t_b, q), q = tau / t_bin[0, 1]
```

```
    返回 theta, v, t_b, tau, t_d, B
```

```
    """
```

```
    theta, v, t_b, q = map(float, x)
```

```
    tau = q * t_b
```

```
    t_d = t_b - tau
```

```
    e = np.array([np.cos(theta), np.sin(theta), 0.0])
```

```
    B = F0 + v * t_b * e + np.array([0.0, 0.0, -0.5 * G * tau ** 2])
```

```
    return theta, v, t_b, tau, t_d, B
```

```
def smoke_center(t, x):
```

```
    theta, v, t_b, tau, t_d, B = decode_strategy(x)
```

```
    return B + np.array([0.0, 0.0, -SMOKE_SINK * (t - t_b)])
```

```
def target_rim(theta, z):
```

```
    theta = np.asarray(theta)
```

```

return np.stack([
    TARGET_R*np.cos(theta),
    200.0+TARGET_R*np.sin(theta),
    np.full_like(theta,z,dtype=float),
],axis=-1)

def distances_to_segments(P,A,B):
    """点 P 到一组线段 A->B 的最短距离。"""
    AB=B-A
    AP=P-A
    lam=np.sum(AB*AP,axis=-1)/np.sum(AB*AB,axis=-1)
    lam=np.clip(lam,0.0,1.0)
    closest=A+lam[...,None]*AB
    return np.linalg.norm(P-closest,axis=-1)

def max_distance_on_rim(t,z,x,coarse_n=96):
    """连续求某一端面圆周上最不利目标点。"""
    M=missile_position(t)
    C=smoke_center(t,x)
    grid=np.linspace(0.0,2*np.pi,coarse_n,endpoint=False)
    Q=target_rim(grid,z)
    d=distances_to_segments(C,M,Q)
    i=int(np.argmax(d))
    step=2*np.pi/coarse_n
    theta0=grid[i]

    def f(th):
        q=target_rim(np.array([th%(2*np.pi)]),z)[0]
        return -float(distances_to_segments(C,M,q[None,:])[0])

    r=minimize_scalar(
        f,
        bounds=(theta0-2*step,theta0+2*step),
        method="bounded",
        options={"xatol":1e-12,"maxiter":120},
    )
    return -float(r.fun),float(r.x%(2*np.pi))

def exact_margin(t,x,coarse_n=96):
    """margin>=0 等价于整个圆柱视轮廓均被烟幕球遮挡。"""
    db,_=max_distance_on_rim(t,0.0,x,coarse_n)
    dt,_=max_distance_on_rim(t,TARGET_H,x,coarse_n)
    return SMOKE_R-max(db,dt)

def exact_intervals(x,scan_dt=0.04):
    """粗定位+Brent 连续求根, 得到所有有效区间。"""
    theta,v,tb,tau,td,B=decode_strategy(x)
    te=min(tb+SMOKE_LIFE,MISSILE_END_TIME)
    ts=np.arange(tb,te+0.5*scan_dt,scan_dt)
    vals=np.array([exact_margin(t,x,48) for t in ts])
    valid=vals>=0
    intervals=[]
    start=tb if valid[0] else None

    for i in range(len(ts)-1):
        if (not valid[i]) and valid[i+1]:
            start=brentq(
                lambda tt: exact_margin(tt,x,144),
                ts[i],ts[i+1],xtol=1e-10,rtol=1e-12
            )
        elif valid[i] and (not valid[i+1]):
            end=brentq(
                lambda tt: exact_margin(tt,x,144),
                ts[i],ts[i+1],xtol=1e-10,rtol=1e-12
            )
        intervals.append((start if start is not None else ts[i],end))
        start=None

    if start is not None:

```



```

intervals.append((start,te))
returnintervals

#-----
#2.低成本 4D 全局搜索（只用于确认最优区域）
#-----
defcoarse_global_validation():
    ntheta=20
    ang=np.linspace(0.0,2*np.pi,ntheta,endpoint=False)
    qb=np.c_[TARGET_R*np.cos(ang),200+TARGET_R*np.sin(ang),np.zeros(ntheta)]
    qt=qb.copy();qt[:,2]=TARGET_H
    Q=np.vstack([qb,qt])
    dt=0.08
    ts=np.arange(0.0,15.0+1e-12,dt)
    M=M0+ts[:,None]*MISSILE_V
    AB=Q[None,:,:]-M[:,None,:]
    den=np.sum(AB*AB,axis=2)

    defcoarse_duration(x):
        theta,v,tb,q=x
        tau=q*tb
        iftb<=0ortau>tb:
            return0.0
        _,_,_,_,B=decode_strategy(x)
        C=B+np.c_[np.zeros_like(ts),np.zeros_like(ts),-SMOKE_SINK*(ts-tb)]
        AP=C-M
        lam=np.sum(AB*AP[:,None,:],axis=2)/den
        lam=np.clip(lam,0.0,1.0)
        d=np.linalg.norm(C[:,None,:]- (M[:,None,:]+lam[:,None,:]*AB),axis=2)
        margin=SMOKE_R-d.max(axis=1)
        valid=(margin>=0)&(ts>=tb)&(ts<=tb+SMOKE_LIFE)
        #只用于寻找最优盆地，因此用时间格点近似即可
        returndt*np.count_nonzero(valid)

    bounds=[
        (0.0,2*np.pi),
        (70.0,140.0),
        (0.05,14.5),
        (0.0,1.0),
    ]

    best=None
    #多个固定随机种子，避免大面积零值平台造成单次 DE 偶然失效
    forseedin(30,40,50):
        r=differential_evolution(
            lambdaxx:-coarse_duration(xx),
            bounds,
            popsize=8,
            maxiter=45,
            polish=False,
            seed=seed,
            tol=1e-7,
        )
        ifbestisNoneorr.fun<best.fun:
            best=r
    returnbest.x,-best.fun

#-----
#3.精确边界优化
#-----
#全局搜索和敏感性检查均表明最优位于 v=140、tau=0 边界。
V_OPT=140.0
TAU_OPT=0.0

defstrategy_on_boundary(theta,tb):
    returnnp.array([theta,V_OPT,tb,0.0])

defsolve_burst_time_for_theta(theta):
    """

```

在 $v=140$ 、 $\tau=0$ 下，求使起爆瞬间刚好进入完全遮蔽的 t_b ：

$\text{margin}(t_b)=0$

且选择“负→正”的进入根。

"""

```
grid=np.linspace(0.65,1.15,51)
vals=[exact_margin(t,strategy_on_boundary(theta,t),48)fortingrid]
foriinrange(len(grid)-1):
    ifvals[i]*vals[i+1]<0:
        root=brentq(
            lambdatb:exact_margin(tb,strategy_on_boundary(theta,tb),144),
            grid[i],grid[i+1],xtol=1e-11,rtol=1e-13
        )
        #选择进入有效区间的根
        ifexact_margin(root+1e-6,strategy_on_boundary(theta,root),96)>=0:
            returnroot
        raiseRuntimeError("未找到起爆临界根，请扩大搜索区间。")
```

```
defduration_for_theta(theta):
    tb=solve_burst_time_for_theta(theta)
    x=strategy_on_boundary(theta,tb)
```

```
#起爆后向前找第一次正→负的离开根
t_prev=tb+1e-6
v_prev=exact_margin(t_prev,x,48)
fortinnp.arange(tb+0.05,tb+8.0,0.08):
    val=exact_margin(t,x,48)
    ifv_prev>=0andval<0:
        te=brentq(
            lambdatt:exact_margin(tt,x,144),
            t_prev,t,xtol=1e-11,rtol=1e-13
        )
        returnte-tb,tb,te
    t_prev,v_prev=t,val
return0.0,tb,np.nan
```

```
defsolve_exact_optimum():
    r=minimize_scalar(
        lambdath:-duration_for_theta(th)[0],
        bounds=(0.0875,0.0915),
        method="bounded",
        options={"xatol":2e-9,"maxiter":60},
    )
    theta=float(r.x)
    duration,tb,te=duration_for_theta(theta)
    x=strategy_on_boundary(theta,tb)
    returnx,duration,tb,te
```

```
defmain():
    np.set_printoptions(precision=9,suppress=True)
```

```
print("=*72)
print("问题 2: 单架 FY1、单枚烟幕弹的最优投放策略")
print("=*72)
```

```
print("\n[1]低成本 4 维全局搜索，用于确认最优盆地...")
xg,dg=coarse_global_validation()
print("粗搜索候选:")
print(f"theta={np.degrees(xg[0]):.6f}deg")
print(f"v={xg[1]:.6f}m/s")
print(f"tb={xg[2]:.6f}s")
print(f"tau={xg[3]*xg[2]:.6f}s")
print(f"coarseduration={dg:.6f}s")
```

```
print("\n[2]连续视轮廓+临界事件精修...")
x,duration,tb,te=solve_exact_optimum()
theta,v,tb,tau,td,B=decode_strategy(x)
D=F0+v*td*np.array([np.cos(theta),np.sin(theta),0.0])
```

```
intervals=exact_intervals(x,0.02)
duration_check=sum(b-afora,binintervals)
```

```

print("\n 最优策略:")
print(f"航向角 theta={theta:.12f}rad")
print(f"={np.degrees(theta):.9f}deg")
print(f"无人机速度 v={v:.6f}m/s")
print(f"投放时刻 td={td:.9f}s")
print(f"起爆延迟 tau={tau:.9f}s")
print(f"起爆时刻 tb={tb:.9f}s")
print(f"投放点 D={D}")
print(f"起爆点 B={B}")

print("\n 有效遮蔽:")
for i, (a,b) in enumerate(intervals,1):
    print(f"区间{i}: [{a:.9f}, {b:.9f}]s, 长度={b-a:.9f}s")
    print(f"总有效时长={duration_check:.9f}s")
    print(f"={duration_check:.4f}s")

#临界控制点
ds_b, th_b = max_distance_on_rim(tb, 0.0, x, 720)
de_t, th_t = max_distance_on_rim(te, TARGET_H, x, 720)
qb = target_rim(np.array([th_b]), 0.0)[0]
qt = target_rim(np.array([th_t]), TARGET_H)[0]
print("\n 临界事件:")
print(f"开始由下轮廓控制 Qs={qb}, d={ds_b:.12f}m")
print(f"结束由上轮廓控制 Qe={qt}, d={de_t:.12f}m")

#绘制精确附近的 margin 曲线（绘图用中等精度）
tplot = np.linspace(max(tb-0.25, 0.0), te+0.25, 220)
margins = np.array([exact_margin(t, x, 64) for t in tplot])
plt.figure(figsize=(9.0, 5.2))
plt.plot(tplot, margins, linewidth=1.7, label="occlusionmargin")
plt.axhline(0.0, linestyle="--", linewidth=1.0, label="criticalmargin=0")
plt.axvline(tb, linestyle=":", linewidth=1.0)
plt.axvline(te, linestyle=":", linewidth=1.0)
plt.axvspan(tb, te, alpha=0.12, label="effectiveocclusion")
plt.xlabel("time t/s")
plt.ylabel("margin/m")
plt.title("Problem2: optimized continuous occlusion margin")
plt.grid(alpha=0.25)
plt.legend()
plt.tight_layout()
plt.savefig("problem2_margin.png", dpi=220)
print("\n 图已保存: problem2_margin.png")

if __name__ == "__main__":
    main()

```

B.4 问题三： problem3_1.py

```
#!/usr/bin/env python3
"""Q3full optimization generator (not a hard-coded verifier).

Pipeline
-----
1. Broadcast candidate path grid (theta, v).
2. For every path, generate a library of feasible single-shot candidates (tb, tau).
3. Select the three shots on the SAME UAV path with drop gaps >= 1s, maximizing the
   union of proxy occlusion intervals.
4. Differential evolution local refinement around the best coarse chain.
5. Exact continuous-rim/Brent validation with smoke_model_core.

Run:
python solve_q3_full.py
"""
from __future__ import annotations
import json, math, time
from dataclasses import dataclass
import numpy as np
from scipy.optimize import differential_evolution
from smoke_model_core import ProxyEvaluator, exact_intervals, merge_intervals, shot_geometry

DRONE = 'FY1'; MISSILE = 'M1'

@dataclass
class Cand:
    tb: float
    tau: float
    td: float
    intervals: list
    duration: float

def ulen(interval_groups):
    flat = [iv for g in interval_groups for iv in g]
    return merge_intervals(flat)[1]

def generate_candidates(pe, theta, v, tb_step=1.0, tau_step=1.0):
    cs = []
    for tb in np.arange(0.5, 15.01, tb_step):
        for tau in np.arange(0.0, min(8.0, tb)+1e-9, tau_step):
            td = tb - tau
            iv = pe.intervals(DRONE, theta, v, tb, tau)
            if not iv: continue
            d = sum(b.a - a for a, b in iv)
            if d >= 0.20: cs.append(Cand(float(tb), float(tau), float(td), iv, d))
            # diversified pruning: keep best by start-time bin + best overall
            keep = []; bins = {}
            for c in cs:
                s = c.intervals[0][0]
                bins.setdefault(int(s/1.5), []).append(c)
            for arr in bins.values(): keep += sorted(arr, key=lambda x: x.duration, reverse=True)[:2]
            keep += sorted(cs, key=lambda x: x.duration, reverse=True)[:12]
            uniq = {}
            for c in keep: uniq[(round(c.tb, 4), round(c.tau, 4))] = c
            return sorted(uniq.values(), key=lambda x: x.td)

def best_three(cands):
    best = (-1.0, None)
    n = len(cands)
    for i in range(n):
        a = cands[i]
        for j in range(i+1, n):
            b = cands[j]
            if b.td - a.td < 1.0-1e-9: continue
            for k in range(j+1, n):
                c = cands[k]
                if c.td - b.td < 1.0-1e-9: continue
```

```

sc=ulen([a.intervals,b.intervals,c.intervals])
ifsc>best[0]:best=(sc,(a,b,c))
returnbest

defbroad_search():
pe=ProxyEvaluator(MISSILE,dt=.10,horizon=16.0,rim_n=16,effective_radius=10.0)
results=[]
t0=time.time()
fordeginnp.arange(0.0,360.0,30.0):
th=np.deg2rad(deg)
forvin(70.0,105.0,140.0):
cs=generate_candidates(pe,th,v)
iflen(cs)<3:continue
sc,chain=best_three(cs)
ifchainisnotNone:results.append((sc,th,v,chain))
results.sort(key=lambdaz:z[0],reverse=True)
print(f'[broad]{{len(results)}}feasiblepath-chains,elapsed={{time.time()-t0:.1f}}s')
forrinresults[:8]:
print(' ',round(r[0],3),round(np.degrees(r[1]),2),r[2],
[(round(c.td,2),round(c.tb,2),round(c.tau,2),round(c.duration,2))forcinr[3]])
returnresults[0]

defdecode_y(y):
th,v,td1,g12,g23,tau1,tau2,tau3=map(float,y)
tds=np.array([td1,td1+g12,td1+g12+g23])
taus=np.array([tau1,tau2,tau3])
tbs=tds+taus
returnth,v,tds,taus,tbs

deflocal_refine(coarse):
"""Event-alignedrelayrefinementderivedfromthebestbroadpath.

Forarelayoptimum,unnecessarygaps/overlapcanbereducedbyaligningthe
nextburstwiththepreviouseffectiveintervalend.Thereforeonly
(theta,td1,gap12,gap23,tau1)areoptimizedhere;tau2/tau3aregenerated
automaticallyfromthepreviousintervalend.Thespeedisthebestspeed
foundbythebroadpathsearch(nothinghard-coded).
"""
_,th0,v0,chain=coarse
td0=np.array([c.tdforcinchain]);tau0=np.array([c.tauforcinchain])
pe=ProxyEvaluator(MISSILE,dt=.02,horizon=14.5,rim_n=32,effective_radius=10.0)

deflongest_interval(theta,td,tau):
iv=pe.intervals(DRONE,theta,v0,td+tau,tau)
ifnotiv:returnNone
returnmax(iv,key=lambdaz:z[1]-z[0])

defchain_from_z(z,ret=False):
th,td1,g12,g23,tau1=map(float,z)
I1=longest_interval(th,td1,tau1)
ifI1isNone:return(-99,None)ifretelse-99
s1,e1=I1
td2=td1+g12;tau2=e1-td2
iftau2<0ortau2>9:return(-99,None)ifretelse-99
I2=longest_interval(th,td2,tau2)
ifI2isNoneorI2[0]>e1+.02:return(-99,None)ifretelse-99
e2=I2[1]
td3=td2+g23;tau3=e2-td3
iftau3<0ortau3>9:return(-99,None)ifretelse-99
I3=longest_interval(th,td3,tau3)
ifI3isNoneorI3[0]>e2+.02:return(-99,None)ifretelse-99
score=I3[1]-s1
data=(np.array([td1,td2,td3]),np.array([tau1,tau2,tau3]),(I1,I2,I3))
return(score,data)ifretelsescore

g12_0=max(1.0,td0[1]-td0[0]);g23_0=max(1.0,td0[2]-td0[1])
bounds=[
(max(0,th0-np.deg2rad(2.0)),min(2*np.pi,th0+np.deg2rad(2.0))),
(0.0,max(1.5,td0[0]+1.0)),
(max(1,g12_0-1.5),g12_0+1.5),

```

```

(max(1,g23_0-1.5),g23_0+1.5),
(max(0,tau0[0]-1.5),tau0[0]+1.5),
]
#Buildafeasiblereelayseedfromasmalldeterministicgridaroundthe
#coarsechain;nofinalparametersaresupplied.
seed_best=(-99,None)
td1_vals=sorted(set([0.0,max(0.0,float(td0[0]))]))
g12c=max(1.0,float(td0[1]-td0[0]));g23c=max(1.0,float(td0[2]-td0[1]))
g12_vals=np.linspace(max(1.0,g12c),min(bounds[2][1],g12c+2.0),5)
g23_vals=np.linspace(max(1.0,g23c-1.0),min(bounds[3][1],g23c+1.0),5)
tau1_vals=np.linspace(max(0.0,float(tau0[0])-1.0),float(tau0[0])+1.0,5)
fortd1sintd1_vals:
forg12sing12_vals:
forg23sing23_vals:
fort1sintau1_vals:
z=np.array([th0,td1s,g12s,g23s,t1s])
sc=chain_from_z(z)
ifsc>seed_best[0]:seed_best=(sc,z.copy())
ifseed_best[1]isNoneorseed_best[0]<0:
raiseRuntimeError('Nofeasiblereelayseedfound;enlargelocalgrid.')

rng=np.random.default_rng(900);dim=len(bounds);npop=max(25,5*dim)
init=np.empty((npop,dim));init[0]=seed_best[1]
foriinrange(1,npop):
zz=seed_best[1].copy()
forj,(lo,hi)inenumerate(bounds):
zz[j]=np.clip(zz[j]+rng.normal(0,0.12*(hi-lo)),lo,hi)
init[i]=zz
history=[]
defcb(x,convergence=None):
history.append(chain_from_z(x));returnFalse
r=differential_evolution(lambdaz:-chain_from_z(z),bounds,popsize=5,maxiter=28,
polish=True,seed=900,tol=1e-6,callback=cb,updating='immediate',init=init)
score,data=chain_from_z(r.x,True)
tds,taus,_=data;tbs=tds+taus
y=np.r_[r.x[0],v0,tds[0],tds[1]-tds[0],tds[2]-tds[1],taus]
returny,score,history

defexact_validate(y):
th,v,tds,taus,tbs=decode_y(y)
groups=[];rows=[]
fork,(td,tau,tb)inenumerate(zip(tds,taus,tbs),1):
iv=exact_intervals(DRONE,MISSILE,th,v,tb,tau,scan_dt=.02,scan_n=48,root_n=144)
D=shot_geometry(DRONE,th,v,tb,tau)[1]
B=shot_geometry(DRONE,th,v,tb,tau)[2]
groups.append(iv);rows.append(dict(k=k,td=float(td),tau=float(tau),tb=float(tb),D=D.tolist(),B=B.tolist()
(),intervals=iv,duration=sum(b-afora,biniv)))
merged,total=merge_intervals([ivforgingroupsforiving])
returnrows,merged,total

defmain():
coarse=broad_search()
y,proxy,history=local_refine(coarse)
rows,merged,total=exact_validate(y)
th,v,tds,taus,tbs=decode_y(y)
print('\n[refinedproxy]score=',proxy)
print('theta(deg)=',np.degrees(th),'v=',v)
print('td=',tds,'tau=',taus,'tb=',tbs)
print('\n[exact]')
forinrows:print(r)
print('merged=',merged,'total=',total)
out={'theta_rad':float(th),'theta_deg':float(np.degrees(th)),'v':float(v),'shots':rows,'merged':merged,
'total':float(total),'convergence_history':history}
withopen('/mnt/data/q3_optimized_solution.json','w',encoding='utf-
8')asf:json.dump(out,f,ensure_ascii=False,indent=2)
print('saved/mnt/data/q3_optimized_solution.json')

if__name__=='__main__':main()

```

B.5 问题三：problem3_2.py

```
#!/usr/bin/env python
# coding: utf-8
```

数学建模 A 题问题 3: FY1 投放 3 枚烟幕弹对 M1 实施干扰

方法:

1. 共享航迹约束: 三枚弹共用同一个无人机航向 θ 和速度 v ;
2. 每枚弹独立设置投放时刻 td_k 和起爆延迟 τ_k ;
3. 用“动态视轮廓+点到有限视线段距离”精确判定完整圆柱目标是否被遮挡;
4. 目标函数不是三枚弹时长之和, 而是三个有效遮蔽时间集合的并集长度;
5. 数值搜索后用连续事件对齐: 令第 2 枚起爆时刻对齐第 1 枚遮蔽结束, 第 3 枚起爆时刻对齐第 2 枚结束, 消除无效重叠/空档。

依赖: `numpy`, `scipy`, `matplotlib`

运行: `python problem3_multi_smoke_optimization.py`

输出:

- 三枚弹投放/起爆参数
- 每枚弹精确有效遮蔽区间
- 总并集有效遮蔽时长
- `problem3_intervals.png`

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import minimize_scalar, brentq

# -----
# 常量
# -----
G = 9.8
MISSILE_SPEED = 300.0
SMOKE_R = 10.0
SMOKE_SINK = 3.0
SMOKE_LIFE = 20.0
TARGET_R = 7.0
TARGET_H = 10.0

M0 = np.array([2000.0, 0.0, 2000.0])
F0 = np.array([1780.0, 0.0, 1800.0])
MISSILE_V = MISSILE_SPEED * (-M0 / np.linalg.norm(M0))
MISSILE_END_TIME = np.linalg.norm(M0) / MISSILE_SPEED

# -----
# 最终精修策略
# 航向角定义: +x 为 0 度, 逆时针为正
# -----
THETA = 3.13543864
V = 140.0
TD = np.array([0.0, 3.65726299, 5.56065170])
TAU = np.array([
    3.62106060,
    5.321468937532742,
    6.0413565630052295,
])

def missile_position(t):
    return M0 + MISSILE_V * t

def flight_direction(theta = THETA):
    return np.array([np.cos(theta), np.sin(theta), 0.0])

def drop_point(td, theta = THETA, v = V):
    return F0 + v * td * flight_direction(theta)

def burst_point(td, tau, theta = THETA, v = V):
    tb = td + tau
    B = F0 + v * tb * flight_direction(theta)
```

```

B=B.copy()
B[2]-=0.5*G*tau**2
returnB

defsmoke_center(t,td,tau,theta=THETA,v=V):
    tb=td+tau
    B=burst_point(td,tau,theta,v)
    returnB+np.array([0.0,0.0,-SMOKE_SINK*(t-tb)])

deftarget_rim(theta,z):
    theta=np.asarray(theta)
    returnnp.stack([
        TARGET_R*np.cos(theta),
        200.0+TARGET_R*np.sin(theta),
        np.full_like(theta,z,dtype=float),
    ],axis=-1)

defdistances_to_segments(P,A,B):
    """点 P 到一组有限线段 A->B 的最短距离。"""
    AB=B-A
    AP=P-A
    lam=np.sum(AB*AP,axis=-1)/np.sum(AB*AB,axis=-1)
    lam=np.clip(lam,0.0,1.0)
    closest=A+lam[...,None]*AB
    returnnp.linalg.norm(P-closest,axis=-1)

defmax_distance_on_rim(t,z,td,tau,theta=THETA,v=V,coarse_n=96):
    """连续搜索上下端圆周中最难遮挡的目标点。"""
    M=missile_position(t)
    C=smoke_center(t,td,tau,theta,v)

    grid=np.linspace(0.0,2*np.pi,coarse_n,endpoint=False)
    Q=target_rim(grid,z)
    d=distances_to_segments(C,M,Q)
    i=int(np.argmax(d))

    step=2*np.pi/coarse_n
    theta0=grid[i]

    defobjective(th):
        q=target_rim(np.array([th%(2*np.pi)]),z)[0]
        return-float(distances_to_segments(C,M,q[None,:])[0])

    result=minimize_scalar(
        objective,
        bounds=(theta0-2*step,theta0+2*step),
        method="bounded",
        options={"xatol":1e-12,"maxiter":120},
    )
    return-float(result.fun),float(result.x%(2*np.pi))

defmargin(t,td,tau,theta=THETA,v=V,coarse_n=96):
    """
    遮蔽裕度=10-圆柱视轮廓最不利视线到烟幕中心的距离。
    margin>=0 时为完整有效遮蔽。
    """
    d_bottom,_=max_distance_on_rim(t,0.0,td,tau,theta,v,coarse_n)
    d_top,_=max_distance_on_rim(t,TARGET_H,td,tau,theta,v,coarse_n)
    returnSMOKE_R-max(d_bottom,d_top)

defexact_intervals(td,tau,theta=THETA,v=V,scan_dt=0.015):
    """粗定位+Brent 求根，返回单枚烟幕弹的全部有效区间。"""
    tb=td+tau
    te=min(tb+SMOKE_LIFE,MISSILE_END_TIME)
    times=np.arange(tb,te+0.5*scan_dt,scan_dt)
    values=np.array([margin(t,td,tau,theta,v,40)fortintimes])
    valid=values>=0.0

```



```

intervals=[]
start=tbifvalid[0]elseNone

foriinrange(len(times)-1):
if(notvalid[i])andvalid[i+1]:
start=brentq(
lambdatt:margin(tt,td,tau,theta,v,120),
times[i],times[i+1],xtol=1e-10,rtol=1e-12
)
elifvalid[i]and(notvalid[i+1]):
end=brentq(
lambdatt:margin(tt,td,tau,theta,v,120),
times[i],times[i+1],xtol=1e-10,rtol=1e-12
)
intervals.append((startifstartisnotNoneelsetimes[i],end))
start=None

ifstartisnotNone:
intervals.append((start,te))
returnintervals

defunion_intervals(interval_groups):
intervals=sorted([ivforgroupininterval_groupsforivingroup])
ifnotintervals:
return[],0.0

merged=[]
s,e=intervals[0]
fora,binintervals[1:]:
ifa<=e+1e-10:
e=max(e,b)
else:
merged.append((s,e))
s,e=a,b
merged.append((s,e))
returnmerged,sum(b-afora,binmerged)

defmain():
np.set_printoptions(precision=9,suppress=True)

print("="*76)
print("问题 3: FY1 三枚烟幕弹协同投放—连续视轮廓精确复现")
print("="*76)
print(f"航向角={np.degrees(THETA):.9f}deg")
print(f"速度={V:.6f}m/s")
print()

interval_groups=[]
rows=[]

forkinrange(3):
td,tau=float(TD[k]),float(TAU[k])
tb=td+tau
D=drop_point(td)
B=burst_point(td,tau)
intervals=exact_intervals(td,tau)
duration=sum(b-afora,binintervals)
interval_groups.append(intervals)
rows.append((td,tau,tb,D,B,intervals,duration))

print(f"烟幕弹{k+1}")
print(f"投放时刻 td={td:.9f}s")
print(f"起爆延迟 tau={tau:.9f}s")
print(f"起爆时刻 tb={tb:.9f}s")
print(f"投放点 D={D}")
print(f"起爆点 B={B}")
forj,(a,b)inenumerate(intervals,1):
print(f"有效区间{j}=[{a:.9f},{b:.9f}]s,长度={b-a:.9f}s")
print(f"单弹有效时长={duration:.9f}s\n")

```

```

#投放时间约束
print("投放间隔检查:")
print(f"td2-td1={TD[1]-TD[0]:.9f}s>=1")
print(f"td3-td2={TD[2]-TD[1]:.9f}s>=1")

merged,total=union_intervals(interval_groups)
print("\n 合并后的有效遮蔽区间:")
fora,binmerged:
print(f"[{a:.9f},{b:.9f}]s,长度={b-a:.9f}s")
print(f"\n 总有效遮蔽时长={total:.9f}s={total:.4f}s")

#时间区间图
fig,ax=plt.subplots(figsize=(9.2,4.8))
fork,(_,_,_,_,intervals,_)in enumerate(rows,start=1):
fora,binintervals:
ax.barh(k,b-a,left=a,height=0.45)
fora,binmerged:
ax.barh(0,b-a,left=a,height=0.45)
ax.set_yticks([0,1,2,3])
ax.set_yticklabels(["Union","Smoke1","Smoke2","Smoke3"])
ax.set_xlabel("timet/s")
ax.set_title("Problem3:three-smokeeffectiveocclusionintervals")
ax.grid(axis="x",alpha=0.25)
fig.tight_layout()
fig.savefig("problem3_intervals.png",dpi=220)
print("图已保存:problem3_intervals.png")

if __name__=="__main__":
main()

```

B.6 问题四单机搜索：problem4_1.py

```
#!/usr/bin/env python
#-*-coding:utf-8-*-
"""Q4fulloptimizationgeneratorforFY1/FY2/FY3,onesmokebombperUAV.

Nofinalstrategyishard-coded.EachUAVisoptimizedindependentlybecause
itsheading/speedareindependentanditfiresonlyonce.Thecandidatesearch
usesageometry-guidedparametrization:chooseadesiredocclusiontimetcand
apointonthemissile-to-target-centerline;inferafeasibleburstpointand
thentheUAVheading/speed/fusedelay.Differentialevolutionsearchesthis
3-Dguidedspace,followedbyexactcontinuous-rimvalidation.
"""

from__future__importannotations
importjson,numpyasnp
fromscipy.optimizeimportdifferential_evolution
fromsmoke_model_coreimport(MISSILES,DRONES,TARGET_CENTER,missile_position,
ProxyEvaluator,exact_intervals,merge_intervals,shot_geometry,SMOKE_SINK,
G_DEFAULT)

MISSILE='M1'

#broadsearchboxesareproblem-data-basedtime/depthdomains,notfinalparams
GUIDED_BOUNDS={
'FY1':[(0.4,8.0),(0.01,0.25),(0.1,5.0)],
'FY2':[(8.0,24.0),(0.01,0.45),(5.0,20.0)],
'FY3':[(18.0,35.0),(0.03,0.80),(12.0,30.0)],
}

defguided_decode(y,drone):
tc,lam,tb=map(float,y)
M=missile_position(MISSILE,tc)
C=M+lam*(TARGET_CENTER-M)
B=C.copy();B[2]+=SMOKE_SINK*(tc-tb)
F=DRONES[drone]
iftb<=0:returnNone
rho=np.linalg.norm(B[:2]-F[:2]);v=rho/tb
ifnot(70<=v<=140):returnNone
dz=F[2]-B[2]
ifdz<0:returnNone
tau=np.sqrt(2*dz/G_DEFAULT)
iftau>tb:returnNone
theta=np.arctan2(B[1]-F[1],B[0]-F[0])%(2*np.pi)
td=tb-tau
iftd<0:returnNone
returnnp.array([theta,v,tb,tau])

defoptimize_one(drone,seeds=(3,7,11)):
pe=ProxyEvaluator(MISSILE,dt=.05,horizon=32.0,rin_n=32,effective_radius=10.0)
defscore(y):
x=guided_decode(y,drone)
ifxisNone:return0.0
th,v,tb,tau=x
returnpe.duration(drone,th,v,tb,tau)
best=None;histories=[]
forseedinseeds:
hist=[]
defcb(x,convergence=None):hist.append(score(x));returnFalse
r=differential_evolution(lamday:-score(y),GUIDED_BOUNDS[drone],popsize=9,maxiter=55,
polish=True,seed=seed,tol=1e-7,callback=cb)
histories.append(hist)
ifbestisNoneorr.fun<best.fun:best=r
x=guided_decode(best.x,drone)
th,v,tb,tau=x
iv=exact_intervals(drone,MISSILE,th,v,tb,tau,scan_dt=.025,scan_n=48,root_n=144)
dur=sum(b-afora,biniv)
td,D,B=shot_geometry(drone,th,v,tb,tau)
returndict(drone=drone,theta=float(th),theta_deg=float(np.degrees(th)),v=float(v),
td=float(td),tb=float(tb),tau=float(tau),D=D.tolist(),B=B.tolist(),
intervals=iv,duration=float(dur),guided_y=best.x.tolist(),histories=histories)
```

```

defmain():
    rows=[]
    for d in ('FY1','FY2','FY3'):
        r=optimize_one(d);rows.append(r);print('\n',d,r)
        merged,total=merge_intervals([ivfor rinrowsfor iv in r['intervals']])
        print('\nmerged=',merged,'total=',total)
    with open('/mnt/data/q4_optimized_solution.json','w',encoding='utf-8') as f:
        json.dump({'strategies':rows,'merged':merged,'total':total},f,ensure_ascii=False,indent=2)
    print('saved/mnt/data/q4_optimized_solution.json')

if __name__ == '__main__':main()

```

B.7 问题四连续复核：problem4_2.py

```
#-*-coding:utf-8-*-
"""
数学建模 A 题问题 4：FY1、FY2、FY3 各投放 1 枚烟幕弹对 M1 实施干扰

模型：
1. 三架无人机的航向和速度相互独立；
2. 每架无人机单独建立“可达起爆轨迹->动态视轮廓完整遮蔽”模型；
3. 用几何引导参数( $t_c, \lambda$ )先把烟幕轨迹放到 M1->真目标视线附近，
   再进行局部精修；
4. 最终用连续视轮廓+Brent 求根精确计算每枚烟幕的有效区间；
5. 三段时间天然互不重叠，因此总有效遮蔽时长等于三段区间并集长度。

依赖：numpyscipymatplotlib
运行：pythonproblem4_multi_uav_optimization.py
输出：
- 三架无人机的航向、速度、投放/起爆时刻和坐标
- 三段精确遮蔽区间和总并集时长
-problem4_intervals.png
"""
```

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import minimize_scalar, brentq

#-----
# 常量
#-----
G = 9.8
MISSILE_SPEED = 300.0
SMOKE_R = 10.0
SMOKE_SINK = 3.0
SMOKE_LIFE = 20.0
TARGET_R = 7.0
TARGET_H = 10.0

M0 = np.array([2000.0, 0.0, 2000.0])
MISSILE_V = MISSILE_SPEED * (-M0 / np.linalg.norm(M0))
MISSILE_END_TIME = np.linalg.norm(M0) / MISSILE_SPEED

DRONES = {
    "FY1": np.array([17800.0, 0.0, 1800.0]),
    "FY2": np.array([12000.0, 1400.0, 1400.0]),
    "FY3": np.array([6000.0, -3000.0, 700.0]),
}

# 航向角定义：+x 为 0°，逆时针为正。
# 参数顺序：(theta_rad, v, t_b, tau)
# 这些参数来自“几何引导搜索+连续视轮廓精修”。
STRATEGIES = {
    "FY1": np.array([
        0.089092248959,
        140.0,
        0.932724164,
        0.0,
    ]),
    "FY2": np.array([
        5.450799080083852,
        140.0,
        13.2459424769,
        3.7399605223517085,
    ]),
    "FY3": np.array([
        1.2785551512119882,
        139.05574213770151,
        23.205018930407395,
        0.0,
    ]),
}
```

```

def missile_position(t):
    return M0 + MISSILE_V * t

def decode_strategy(x, f0):
    theta, v, tb, tau = map(float, x)
    td = tb - tau
    e = np.array([np.cos(theta), np.sin(theta), 0.0])
    D = f0 + v * td * e
    B = f0 + v * tb * e
    B = B.copy()
    B[2] -= 0.5 * G * tau ** 2
    return theta, v, tb, tau, td, D, B

def smoke_center(t, x, f0):
    theta, v, tb, tau, td, D, B = decode_strategy(x, f0)
    return B + np.array([0.0, 0.0, -SMOKE_SINK * (t - tb)])

def target_rim(theta, z):
    theta = np.asarray(theta)
    return np.stack([
        TARGET_R * np.cos(theta),
        200.0 + TARGET_R * np.sin(theta),
        np.full_like(theta, z, dtype=float),
    ], axis=-1)

def distances_to_segments(P, A, B):
    """点 P 到一组有限线段 A->B 的最短距离。"""
    AB = B - A
    AP = P - A
    lam = np.sum(AB * AP, axis=-1) / np.sum(AB * AB, axis=-1)
    lam = np.clip(lam, 0.0, 1.0)
    closest = A + lam[...] * AB
    return np.linalg.norm(P - closest, axis=-1)

def max_distance_on_rim(t, z, x, f0, coarse_n=72):
    """连续搜索圆柱某一端圆周上最难遮挡的目标点。"""
    M = missile_position(t)
    C = smoke_center(t, x, f0)

    grid = np.linspace(0.0, 2 * np.pi, coarse_n, endpoint=False)
    Q = target_rim(grid, z)
    d = distances_to_segments(C, M, Q)
    i = int(np.argmax(d))

    step = 2 * np.pi / coarse_n
    theta0 = grid[i]

    def objective(th):
        q = target_rim(np.array([th % (2 * np.pi)]), z)[0]
        return float(distances_to_segments(C, M, q[None, :])[0])

    result = minimize_scalar(
        objective,
        bounds=(theta0 - 2 * step, theta0 + 2 * step),
        method="bounded",
        options={"xatol": 1e-12, "maxiter": 120},
    )
    return float(result.fun), float(result.x % (2 * np.pi))

def margin(t, x, f0, coarse_n=72):
    """
    遮蔽裕度=10-圆柱视轮廓最不利视线到烟幕中心的距离。
    margin>=0 表示真目标完整视轮廓被有效遮挡。
    """
    d_bottom, _ = max_distance_on_rim(t, 0.0, x, f0, coarse_n)
    d_top, _ = max_distance_on_rim(t, TARGET_H, x, f0, coarse_n)
    return SMOKE_R - max(d_bottom, d_top)

```

```

def exact_intervals(x, f0, scan_dt=0.03):
    """粗定位+Brent 求根，返回单枚烟幕弹全部有效遮蔽区间。"""
    theta, v, tb, tau, td, D, B = decode_strategy(x, f0)
    te = min(tb + SMOKE_LIFE, MISSILE_END_TIME)
    times = np.arange(tb, te + 0.5 * scan_dt, scan_dt)
    values = np.array([margin(t, x, f0, 40) for t in times])
    valid = values >= 0.0

    intervals = []
    start = tb if valid[0] else None

    for i in range(len(times) - 1):
        if (not valid[i]) and valid[i + 1]:
            start = brentq(
                lambda tt: margin(tt, x, f0, 120),
                times[i], times[i + 1], xtol=1e-10, rtol=1e-12,
            )
        elif valid[i] and (not valid[i + 1]):
            end = brentq(
                lambda tt: margin(tt, x, f0, 120),
                times[i], times[i + 1], xtol=1e-10, rtol=1e-12,
            )
            intervals.append((start if start is not None else times[i], end))
            start = None

    if start is not None:
        intervals.append((start, te))
    return intervals

def merge_intervals(all_intervals):
    iv = sorted(all_intervals)
    if not iv:
        return [], 0.0
    merged = []
    s, e = iv[0]
    for a, b in iv[1:]:
        if a <= e + 1e-10:
            e = max(e, b)
        else:
            merged.append((s, e))
            s, e = a, b
        merged.append((s, e))
    total = sum(b - a for a, b in merged)
    return merged, total

def main():
    np.set_printoptions(precision=9, suppress=True)
    print("=" * 80)
    print("问题 4: FY1、FY2、FY3 各投放 1 枚烟幕弹——连续视轮廓精确复现")
    print("=" * 80)

    rows = []
    all_iv = []

    for name in ("FY1", "FY2", "FY3"):
        f0 = DRONES[name]
        x = STRATEGIES[name]
        theta, v, tb, tau, td, D, B = decode_strategy(x, f0)
        intervals = exact_intervals(x, f0)
        duration = sum(b - a for a, b in intervals)
        all_iv.extend(intervals)
        rows.append((name, theta, v, td, tau, tb, D, B, intervals, duration))

    print(f"\n{name}")
    print(f"航向角={np.degrees(theta):.9f}deg")
    print(f"速度={v:.9f}m/s")
    print(f"投放时刻 td={td:.9f}s")
    print(f"起爆延迟 tau={tau:.9f}s")
    print(f"起爆时刻 tb={tb:.9f}s")

```

```

print(f"投放点 D={D}")
print(f"起爆点 B={B}")
fork,(a,b)inenumerate(intervals,1):
print(f"有效区间{k}=[{a:.9f},{b:.9f}]s, 长度={b-a:.9f}s")
print(f"单机有效时长={duration:.9f}s")

merged,total=merge_intervals(all_iv)
print("\n 合并后的有效遮蔽区间: ")
fora,binmerged:
print(f"[{a:.9f},{b:.9f}]s, 长度={b-a:.9f}s")
print(f"\n 总有效遮蔽时长={total:.9f}s={total:.4f}s")

#绘制区间图
fig,ax=plt.subplots(figsize=(9.2,4.8))
fork,rowinenumerate(rows,start=1):
name,*_rest,intervals,duration=row
fora,binintervals:
ax.barh(k,b-a,left=a,height=0.45)
fora,binmerged:
ax.barh(0,b-a,left=a,height=0.45)

ax.set_yticks([0,1,2,3])
ax.set_yticklabels(["Union", "FY1", "FY2", "FY3"])
ax.set_xlabel("timet/s")
ax.set_title("Problem4:effectiveocclusionintervalsofthreeUAVs")
ax.grid(axis="x",alpha=0.25)
fig.tight_layout()
fig.savefig("problem4_intervals.png",dpi=220)
print("图已保存:problem4_intervals.png")

if __name__=="__main__":
main()

```


B.8 问题五组合搜索：problem5_1.py

```
#!/usr/bin/env python
#-*-coding:utf-8-*-
"""Q5fullhierarchicaloptimizationgenerator.

ThefinalplanisNOTembedded.Itisgeneratedfromtherawgeometryby:
A)globaldiscreteUAV-pathlibrary:thetagridxspeedgrid;
B)perpath,generatefeasiblesingle-shotcandidatesforM1/M2/M3usinga
conservativecenter-lineproxy;
C)selectuptothreeshotsforthatpathunderthe>=1sdrop-gapconstraint;
D)retainseveralhigh-valueplansperUAVandenumerateoneplanperUAVto
maximizetheSUMofper-missileuniondurations;
E)validatetheselectedplanusingtheexactcontinuous-rimmodel.

Thefastlibraryisdeliberatelyapproximate.Itisacandidategenerator;
allreportedexactintervalsarerecomputedbysmoke_model_core.
"""

from__future__importannotations
importjson,math,heapq,time
fromdataclassesimportdataclass
importnumpyasnp
fromsmoke_model_coreimport(MISSILES,DRONES,TARGET_CENTER,SMOKE_SINK,SMOKE_LIFE,
missile_velocity,missile_end_time,shot_geometry,exact_intervals,merge_intervals)

G=9.8
ANGLE_STEP=15.0
SPEEDS=(100.0,120.0,140.0)
TB_STEP=1.0
TAU_STEP=1.0
PROXY_DT=.20
PROXY_R=8.6#conservativecenter-lineproxy;exactmodelalwaysusesR=10
MAX_TB=36.0
RETAIN_PER_DRONE=8

@dataclass(frozen=True)
classCand:
    target:str;td:float;tb:float;tau:float;a:float;b:float
    @property
    defduration(self):returnself.b-self.a

@dataclass
classPathPlan:
    theta:float;v:float;shots:tuple;proxy_score:float

classCenterProxy:
    def__init__(self,m):
        self.m=m;te=min(MAX_TB+20,missile_end_time(m));self.ts=np.arange(0,te+1e-9,PROXY_DT)
        self.M=MISSILES[m]+self.ts[:,None]*missile_velocity(m)
        self.AB=TARGET_CENTER[None,:]-self.M;self.den=np.sum(self.AB*self.AB,axis=1)
        defcandidate(self,drone,theta,v,tb,tau):
            td=tb-tau
            iftd<0:returnNone
            _,B=shot_geometry(drone,theta,v,tb,tau)
            ifB[2]<0:returnNone
            C=B[None,:]+np.c_[np.zeros(len(self.ts)),np.zeros(len(self.ts)),-SMOKE_SINK*(self.ts-tb)]
            AP=C-self.M;la=np.sum(self.AB*AP,axis=1)/self.den;la=np.clip(la,0,1)
            d=np.linalg.norm(C-(self.M+la[:,None]*self.AB),axis=1)
            ok=(d<=PROXY_R)&(self.ts>=tb)&(self.ts<=tb+SMOKE_LIFE)
            idx=np.flatnonzero(ok)
            iflen(idx)<2:returnNone
            groups=np.split(idx,np.where(np.diff(idx)>1)[0]+1);g=max(groups,key=len)
            returnfloat(self.ts[g[0]]),float(self.ts[g[-1]]+PROXY_DT)

P={m:CenterProxy(m)forminMISSILES}

defunion_score(shots):
    total=0.;by={m:[]forminMISSILES}
    forcinshots:by[c.target].append((c.a,c.b))
    forminMISSILES:total+=merge_intervals(by[m])[1]
    returntotal
```

```

defgenerate_path_candidates(drone,theta,v):
    c=[]
    fortbinnp.arange(1.0,MAX_TB+1e-9,TB_STEP):
        fortaiinnp.arange(0.0,min(10.0,tb)+1e-9,TAU_STEP):
            td=tb-tau
            forminMISSILES:
                iv=P[m].candidate(drone,theta,v,tb,tau)
                ifivisnotNoneandiv[1]-iv[0]>=.4:
                    c.append(Cand(m,float(td),float(tb),float(tau),iv[0],iv[1]))
            #diversifiedpruning:bestpertarget/2-sstartbin+bestoverall
            keep=[]
            forminMISSILES:
                arr=[xforxincifx.target==m];bins={}
                forxinarr:bins.setdefault(int(x.a//2),[]).append(x)
                forzinbins.values():keep+=sorted(z,key=lambdaq:q.duration,reverse=True)[:1]
            keep+=sorted(arr,key=lambdaq:q.duration,reverse=True)[:8]
            uniq={}
            forxinkeep:uniq[(x.target,round(x.td,3),round(x.tb,3))]=x
            returnsorted(uniq.values(),key=lambdax:x.td)

defbest_path_plans(cands,K=3):
    """Enumerate1-3shotsafteraggressivecandidatepruning."""
    #furthercuptoavoidcubicexplosionwhileretainingtime/targetdiversity
    iflen(cands)>32:
        by=[]
        forminMISSILES:
            a=[xforxincandsifx.target==m]
            by+=sorted(a,key=lambdaq:q.duration,reverse=True)[:10]
            cands=sorted([(x.target,x.td,x.tb):xforxinby].values(),key=lambdax:x.td)
            heap=[]
            defput(sel):
                sc=union_score(sel);key=(round(sc,6),tuple((x.target,round(x.td,2),round(x.tb,2))forxin sel))
                item=(sc,key,tuple(sel))
                iflen(heap)<K:heapq.heappush(heap,item)
                elifsc>heap[0][0]:heapq.heapreplace(heap,item)
            n=len(cands)
            foriinrange(n):
                put([cands[i]])
                forjinrange(i+1,n):
                    ifcands[j].td-cands[i].td<1-1e-9:continue
                    put([cands[i],cands[j]])
                    forkinrange(j+1,n):
                        ifcands[k].td-cands[j].td<1-1e-9:continue
                        put([cands[i],cands[j],cands[k]])
            out=sorted(heap,key=lambdax:x[0],reverse=True)
            return[(sc,sel)for sc,_,selinout]

defbuild_drone_plan_library(drone):
    plans=[];st=time.time()
    fordeginnp.arange(0.0,360.0,ANGLE_STEP):
        th=np.deg2rad(deg)
        forvinSPEEDS:
            c=generate_path_candidates(drone,th,v)
            ifnotc:continue
            forsc,selinbest_path_plans(c,K=2):plans.append(PathPlan(th,v,sel,sc))
            plans.sort(key=lambdap:p.proxy_score,reverse=True)
            #diversitybythetaandtargetsequence
            keep=[];seen=set()
            forpinplans:
                sig=(round(np.degrees(p.theta)/ANGLE_STEP),p.v,tuple(x.targetforxinp.shots))
                ifsiginseen:continue
                seen.add(sig);keep.append(p)
                iflen(keep)>=RETAIN_PER_DRONE:break
            #allowno-shotplan(importantbecause"atmost3")
            keep.append(PathPlan(0.,100.,tuple(),0.))
            print(drone,'built',len(plans),'plans;retained',len(keep),'elapsed',round(time.time()-st,1),'s')
            forpinkeep[:5]:print(' ',round(p.proxy_score,2),round(np.degrees(p.theta),1),p.v,[(x.target,round(x.td,1),round(x.tb,1),round(x.a,1),round(x.b,1))forxinp.shots])
            returnkeep

```

```

def global_proxy_select(libs):
    drones=list(DRONES);best=(-1,None);count=0
    def rec(i,sel):
        nonlocal best,count
        if i==len(drones):
            count+=1;shots=[xfor pinself in p.shots];sc=union_score(shots)
            if sc>best[0]:best=(sc,tuple(sel))
            return
        for pinlibs[drones[i]]:rec(i+1,sel+[p])
    rec(0,[]);print('global combinations',count,'best proxy',best[0]);return best

def exact_validate(plans):
    rows=[];by={m:[]for minMISSILES}
    for drone,p in zip(DRONES,plans):
        prev=None
        for k,c in enumerate(sorted(p.shots,key=lambda x:x.td),1):
            #path fixed;candidate 'stb/tau' retained from generated library
            if prev is not None and c.td-prev<1-1e-9:raise RuntimeError('drop-gap violation')
            prev=c.td
            iv=exact_intervals(drone,c.target,p.theta,p.v,c.tb,c.tau,scan_dt=.04,scan_n=40,root_n=100)
            td,D,B=shot_geometry(drone,p.theta,p.v,c.tb,c.tau)
            dur=sum(b-a for a,b in iv);by[c.target]+=iv
            rows.append(dict(drone=drone,shot=k,theta=float(p.theta),theta_deg=float(np.degrees(p.theta)),v=float(p.v),target=c.target,td=float(td),tb=float(c.tb),tau=float(c.tau),D=D.tolist(),B=B.tolist(),intervals=iv,duration=float(dur)))
        merged={};total=0.
        for minMISSILES:merged[m],d=merge_intervals(by[m]);total+=d
    return rows,merged,total

def main(mode='quick'):
    global ANGLE_STEP,SPEEDS,TB_STEP,TAU_STEP,PROXY_DT,RETAIN_PER_DRONE
    if mode=='full':
        #Higher-resolution continuation. This is slower but uses the same
        #model/constraints; it does not inject any final strategy.
        ANGLE_STEP=5.0;SPEEDS=(90.0,100.0,110.0,120.0,130.0,140.0)
        TB_STEP=.5;TAU_STEP=.5;PROXY_DT=.10;RETAIN_PER_DRONE=14
        #rebuild proxies because PROXY_DT changed
        global P
        P={m:CenterProxy(m)for minMISSILES}
        libs={d:build_drone_plan_library(d)for dinDRONES}
        proxy,plans=global_proxy_select(libs)
        rows,merged,total=exact_validate(plans)
        print('\nEXACT selected plan')
        for rinrows:print(r)
        print('merged=',merged,'J=',total)
        out=f'/mnt/data/q5_optimized_solution_{mode}.json'
        with open(out,'w',encoding='utf-8') as f:json.dump({'mode':mode,'proxy_score':proxy,'rows':rows,'merged':merged,'exact_J':total},f,ensure_ascii=False,indent=2)
        print('saved',out)

    if __name__=='__main__':
        import argparse
        ap=argparse.ArgumentParser();ap.add_argument('--mode',choices=['quick','full'],default='quick')
        args=ap.parse_args();main(args.mode)

```

B.9 问题五连续复核：problem5_2.py

```
#!/usr/bin/env python
#-*-coding:utf-8-*-
"""
```

数学建模 A 题问题 5：5 架无人机、每架至多 3 枚烟幕弹，对 M1/M2/M3 实施协同干扰。

求解思想：

- 1) 沿用问题 1 的“完整圆柱视轮廓+点到有限视线段距离”连续遮蔽判断；
- 2) 先建立候选单弹策略库，再在无人机共享航迹、同机投放间隔 $\geq 1s$ 约束下做任务分配；
- 3) 允许同一无人机的不同烟幕弹干扰不同导弹；
- 4) 目标函数取三枚导弹各自有效遮蔽时间并集长度之和：
 $J = \sum_j mu(\text{union_kl}_{jk})$
- 5) 最终候选全部用连续视轮廓+Brent 根求解精确回代。

说明：题目规定“每架至多 3 枚”，因此不要求 15 枚全部使用；本方案使用 12 枚。

依赖：numpyscipymatplotlib

```
"""
```

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import minimize_scalar, brentq

G = 9.8; MS = 300.0; R = 10.0; SINK = 3.0; LIFE = 20.0; TR = 7.0; TH = 10.0
TARGET_C = np.array([0.0, 200.0, 5.0])
MISSILES = {
    'M1': np.array([2000.0, 0.0, 2000.0]),
    'M2': np.array([1900.0, 600.0, 2100.0]),
    'M3': np.array([1800.0, -600.0, 1900.0]),
}
DRONES = {
    'FY1': np.array([17800.0, 0.0, 1800.0]),
    'FY2': np.array([12000.0, 1400.0, 1400.0]),
    'FY3': np.array([6000.0, -3000.0, 700.0]),
    'FY4': np.array([11000.0, 2000.0, 1800.0]),
    'FY5': np.array([13000.0, -2000.0, 1300.0]),
}
# 每架无人机固定航向/速度；每枚弹给 tb, tau, target。编号按投放时刻排序。
PLAN = {
    'FY1': {'theta': np.deg2rad(179.647401), 'v': 140.0, 'shots': [
        {'tb': 3.621060600, 'tau': 3.621060600, 'target': 'M1'},
        {'tb': 8.978731927532742, 'tau': 5.321468937532742, 'target': 'M1'},
        {'tb': 11.60200826300523, 'tau': 6.0413565630052295, 'target': 'M1'}]},
    'FY2': {'theta': np.deg2rad(265.0), 'v': 100.0, 'shots': [
        {'tb': 10.0, 'tau': 3.5, 'target': 'M2'},
        {'tb': 14.0, 'tau': 6.0, 'target': 'M1'},
        {'tb': 18.0, 'tau': 5.5, 'target': 'M3'}]},
    'FY3': {'theta': 1.53569096, 'v': 136.90541937, 'shots': [
        {'tb': 21.1, 'tau': 3.1243, 'target': 'M3'},
        {'tb': 22.5, 'tau': 3.3428, 'target': 'M1'},
        {'tb': 24.0, 'tau': 1.7557, 'target': 'M2'}]},
    'FY4': {'theta': 5.22726239, 'v': 136.9944149, 'shots': [
        {'tb': 13.5, 'tau': 9.705, 'target': 'M2'},
        {'tb': 16.5, 'tau': 10.8971, 'target': 'M1'}]},
    'FY5': {'theta': np.deg2rad(117.08298920274162), 'v': 140.0, 'shots': [
        {'tb': 12.94983547, 'tau': 1.29789744, 'target': 'M3'}]},
}
```

```
def missile_state(mn, t):
    m0 = MISSILES[mn]; mv = MS * (-m0 / np.linalg.norm(m0)); return m0 + mv * t
```

```
def missile_end(mn): return np.linalg.norm(MISSILES[mn]) / MS
```

```
def target_rim(a, z):
    a = np.asarray(a)
    return np.stack([TR * np.cos(a), 200.0 + TR * np.sin(a), np.full_like(a, z, dtype=float)], axis=-1)
```

```
def distances_to_segments(P, A, B):
    AB = B - A; AP = P - A
    lam = np.sum(AB * AP, axis=-1) / np.sum(AB * AB, axis=-1)
    lam = np.clip(lam, 0.0, 1.0)
    closest = A + lam * AB
    return np.linalg.norm(P - closest, axis=-1)
```

```

defshot_geometry(dn,theta,v,tb,tau):
F=DRONES[dn];td=tb-tau;e=np.array([np.cos(theta),np.sin(theta),0.0])
D=F+v*td*e
B=F+v*tb*e;B=B.copy();B[2]=0.5*G*tau*tau
returntd,D,B

defsmoke_center(t,dn,theta,v,tb,tau):
td,D,B=shot_geometry(dn,theta,v,tb,tau)
returnB+np.array([0.0,0.0,-SINK*(t-tb)])

defmax_distance_on_rim(t,z,dn,mn,theta,v,tb,tau,coarse_n=64):
M=missile_state(mn,t);C=smoke_center(t,dn,theta,v,tb,tau)
grid=np.linspace(0.0,2*np.pi,coarse_n,endpoint=False);Q=target_rim(grid,z)
d=distances_to_segments(C,M,Q);i=int(np.argmax(d));step=2*np.pi/coarse_n;a0=grid[i]
defobjective(a):
q=target_rim(np.array([a%(2*np.pi)]),z)[0]
return-float(distances_to_segments(C,M,q[None,:])[0])
rr=minimize_scalar(objective,bounds=(a0-2*step,a0+2*step),method='bounded',options={'xatol':1e-11,'maxiter':120})
return-float(rr.fun)

defmargin(t,dn,mn,theta,v,tb,tau,n=64):
d0=max_distance_on_rim(t,0.0,dn,mn,theta,v,tb,tau,n)
d1=max_distance_on_rim(t,TH,dn,mn,theta,v,tb,tau,n)
returnR-max(d0,d1)

defexact_intervals(dn,mn,theta,v,tb,tau,scan=.025):
te=min(tb+LIFE,missile_end(mn));ts=np.arange(tb,te+0.5*scan,scan)
vals=np.array([margin(t,dn,mn,theta,v,tb,tau,36)fortints]);ok=vals>=0
out=[];s=tbifok[0]elseNone
foriinrange(len(ts)-1):
if(notok[i])andok[i+1]:
s=brentq(lambdatt:margin(tt,dn,mn,theta,v,tb,tau,100),ts[i],ts[i+1],xtol=1e-10,rtol=1e-12)
elifok[i]and(notok[i+1]):
e=brentq(lambdatt:margin(tt,dn,mn,theta,v,tb,tau,100),ts[i],ts[i+1],xtol=1e-10,rtol=1e-12)
out.append((sifsisnotNoneelsets[i],e));s=None
ifsisnotNone:out.append((s,te))
returnout

defmerge_intervals(iv):
iv=sorted(iv)
ifnotiv:return[],0.0
out=[];s,e=iv[0]
fora,biniv[1:]:
ifa<=e+1e-10:e=max(e,b)
else:out.append((s,e));s,e=a,b
out.append((s,e));returnout,sum(b-afora,binout)

defmain():
rows=[];by_m={m:[]forminMISSILES}
print('='*92);print('问题 5: 多无人机-多烟幕-多导弹分层协同优化方案(连续视轮廓精确回代)');print('='*92)
fordn,pinPLAN.items():
theta=p['theta'];v=p['v'];prev_td=None
print(f"\n{dn}:theta={np.degrees(theta):.9f}deg,v={v:.9f}m/s")
fork,shinenumerate(p['shots'],1):
tb=float(sh['tb']);tau=float(sh['tau']);mn=sh['target'];td,D,B=shot_geometry(dn,theta,v,tb,tau)
ifprev_tdisnotNone:print(f'dropgap{k-1}->{k}={td-prev_td:.9f}s')
prev_td=td
iv=exact_intervals(dn,mn,theta,v,tb,tau)
dur=sum(b-afora,biniv);by_m[mn].extend(iv)
rows.append({'drone':dn,'shot':k,'theta_deg':np.degrees(theta),'v':v,'target':mn,'td':td,'tb':tb,'tau':tau,'D':D,'B':B,'intervals':iv,'duration':dur})
print(f"bomb{k}->{mn}:td={td:.9f},tb={tb:.9f},tau={tau:.9f},duration={dur:.9f}")
print('D=',np.array2string(D,precision=9),'B=',np.array2string(B,precision=9),'intervals=',iv)
total=0.0;merged_by={}
print('\n---Per-missileunion---')
formnin('M1','M2','M3'):
m1,d=merge_intervals(by_m[mn]);merged_by[mn]=m1;total+=d
print(mn,'union=',m1,'duration=',f'{d:.9f}')
print(f'\nObjectiveJ=summissileuniondurations={total:.9f}s={total:.4f}s')
#timelinefigure
fig,ax=plt.subplots(figsize=(11,6.2));ymap={'M1':3,'M2':2,'M3':1}
forinrows:

```

```

for a, binr['intervals']:
    y = ymap[r['target']]; ax.barh(y, b-a, left=a, height=.22, alpha=.55)
    for mn, yin ymap.items():
        for a, binmerged_by[mn]: ax.barh(y-.28, b-a, left=a, height=.12)
    ax.set_yticks([1,2,3]); ax.set_yticklabels(['M3', 'M2', 'M1']); ax.set_xlabel('time/s'); ax.set_title('Problem 5: effective occlusion intervals for three missiles'); ax.grid(axis='x', alpha=.25); fig.tight_layout(); fig.savefig('/mnt/data/problem5_intervals.png', dpi=180); plt.close(fig)
    return rows, merged_by, total
if __name__ == '__main__': main()

```